LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

The whole project: what we built, what happened, and why
Study guide for progress meeting 2 (CS-5187 Term Project A3)

Venu, Bhuvan, Gowtham

Status: 24 October 2026

How to use this document

Read Part I (one page) and the numbers cheat sheet (Section 2) first: together they answer most questions in a meeting. Parts II–VI explain every component in depth: what it does, why it is built this way, and what it was measured to do. Part VII is a long list of questions the professor may ask, each with a short answer we can give out loud. Every number in this document comes from a run recorded in the repository (path given) or is marked *planned*.

Contents

I The project in brief **5**

1 The project on one page **5**

2 Numbers cheat sheet **6**

3 Team and grading **7**

4 What happened, in order **7**

4.1 Before meeting 1 (until 16 October) . . . 7
4.2 Meeting 1 (16 October) . . . 8
4.3 The bug hunt (16–17 October) . . . 8
4.4 The build session (23 October) . . . 9
4.5 The plan-coverage round (24 October) . . . 10

II Background **10**

5 Concepts we rely on **10**

5.1 How PostgreSQL runs a query . . . 10
5.2 The test-oracle problem and how we answer it . . . 11
5.3 Fuzzing background . . . 11
5.4 Reduction, benchmarks, statistics . . . 12

5.5 Course lectures used 12

III The system **12**

6 Architecture **13**
6.1 Repository map 14
6.2 One trial 14

7 Step 1: the typed grammar and the generator **14**
7.1 The `.pgg` format 14
7.2 The engine 15
7.3 The generator 15
7.4 The schema 16

8 Steps 2–3: executing a test case and the oracles **16**
8.1 The executor 16
8.2 Classification 17
8.3 Determinism: which queries may be compared at all 18
8.4 Result digests 18
8.5 Oracle 1: plan variants (differential testing inside one server) 19
8.6 Oracle 2: metamorphic rewrites of the derivation tree 19
8.7 Oracle 3: the RLS leak oracle (information flow) 21
8.8 Oracle 4: CERT (cardinality-estimation restriction testing) 22
8.9 Oracle 5: cross-version (master vs 18.6) 22
8.10 Signatures (deduplication) 22

9 Step 4: triage – from a failure to a bug report **23**
9.1 Reproduce 23
9.2 Reduction, three levels 23
9.3 Observe and localize 24
9.4 Outputs 24

10 The precision ledger (RQ2 and claim 2) **25**

11 Step 5: the LLM arm – the LLM proposes, the server decides **26**
11.1 Why an LLM at all, and why this way 26
11.2 The pipeline 27
11.3 The validation gate 27
11.4 The rewrite gate (LLM-proposed metamorphic rewrites) 28
11.5 Leakage and cost 29
11.6 RQ4: what we will report 29

IV Evaluation **29**

12 The re-injected bug benchmark (RQ3) **29**
12.1 Why we need it . . . 29
12.2 Selection . . . 29
12.3 Servers and verification . . . 29
12.4 Attribution . . . 30

13 Experiment design **31**
13.1 Research questions . . . 31
13.2 Arms . . . 31
13.3 Metrics . . . 31
13.4 Statistics . . . 32

14 Results so far **32**
14.1 Final campaigns (29 Oct): RQ1–RQ4 . . . 32
14.2 BUG-01: JSON_TABLE does not assign collations . . . 33
14.3 BUG-02: range containment with a NULL element . . . 34
14.4 Other candidates from the bug hunt . . . 35
14.5 Smoke run 2 (RQ3 pipeline, 15 minutes per arm, injected server) . . . 35
14.6 Full-system smoke on master (3 minutes, one worker) . . . 35
14.7 Component checks . . . 35
14.8 Classes of false positives we found and fixed . . . 36

V Paper, plan and demo **36**

15 The paper **36**
15.1 The five graded parts and where they are . . . 37
15.2 Contributions as we state them . . . 37

16 Execution plan and timeline **37**

17 Live demo script **38**

VI Questions the professor may ask **39**

18 Questions and answers **39**
18.1 Motivation and novelty . . . 39
18.2 Grammar and generator . . . 40
18.3 Oracles . . . 41
18.4 Triage and bugs . . . 42
18.5 LLM arm . . . 43
18.6 Evaluation and statistics . . . 44

18.7 Course connection and process 45

VII Appendix **45**

A Glossary **45**

B Commands **47**

C File index **48**

Part I
The project in brief

1 The project on one page

The problem. PostgreSQL is the most widely used open-source relational database. Its query planner and executor change every week on the development branch (*master*). The most harmful bugs are *wrong results*: a query returns the wrong rows, without an error, without a crash, without any trace in the logs. Fuzzers for database systems mostly find crashes; tools that find wrong results (SQLancer: TLP, NoREC, PQS) use small, hand-written query generators. Neither line of work knows what changed in the code yesterday.

Our idea in one sentence. Generate SQL from a *typed grammar* and keep every generated query as a *derivation tree*; use that tree for four jobs (generation, mutation, metamorphic oracles, and reduction of failures); and let an LLM read recent PostgreSQL commits and *extend* the grammar and the oracle, while a validation gate on a real server decides what the LLM output is allowed to change.

The experiment (backbone of the paper). Three arms under the same budget, server, schema and oracles: **Random/Mutation** (token-level mutation of SQL seeds) → **Grammar** (typed derivation trees) → **Grammar+LLM** (grammar extended from recent commits). Four research questions: RQ1 reach (validity, coverage, features, plan shapes), RQ2 bugs and oracle precision, RQ3 time to find 23 real bugs that we put back into PostgreSQL, RQ4 what the LLM adds and costs.

Where we are (24 Oct).

- Every item of the meeting-1 plan is implemented and tested on a real PostgreSQL master server (details: `docs/PLAN_COVERAGE.md`); 148 unit tests pass; about 10,500 lines of Python plus a 628-line grammar.
- **Two previously unknown PostgreSQL bugs found**, root-caused, with patches and regression tests: **BUG-01** (JSON_TABLE collation: assertion failure on master, wrong result and spurious errors on 18.6) and **BUG-02** (range containment with a NULL element returns `true`/`false` instead of NULL in PostgreSQL 17, 18.6 and master). Filing upstream is the next step.
- A benchmark of **23 real bugs** re-injected into one server by reverting their fixes: 20 observable, only 9 of them visible to a crash-only fuzzer.
- Smoke runs work end to end; the long campaigns (RQ1–RQ4) are ready to run from `docs/EXECUTION_PLAN.md` (about 1 h of builds, then two overnight runs).
- The paper (ACM/FSE format) is drafted with all five graded parts; the result numbers are placeholders until the campaigns finish.

The three claims (from the meeting-1 slides) and their status

1. **One tree, four jobs.** The derivation tree generates, mutates, is rewritten by the metamorphic oracles, and is reduced in triage. *All four are in code.* Tree reduction made a real BUG-01 reproducer 79 tokens long where token reduction alone reached 116.
2. **Soundness we measure, instead of claiming it.** Generator soundness = valid-statement rate (99.5–99.7%). Oracle soundness = triage precision per oracle, before and after each false-positive class was fixed (Section 10).
3. **The LLM proposes; the server decides.** Nothing the LLM writes enters the fuzzer unless it parses, is derived, runs, reaches the changed lines of the commit, and, for oracle rewrites, never changes a result on concrete values and on 150 live test cases.

2 Numbers cheat sheet

What	Value (source)
Subject	PostgreSQL master `26bd08fb` (16 Oct 2026), `-enable-cassert`; release reference 18.6
Grammar	78 rules, 108 variants, 421 alternatives, 13 SQL types, 628 lines (`src/grammar/postgres.pgg`)
Schema	9 relations: plain, partitioned (`tp`, `tl`), inheritance (`ti`), view (`v0`), generated columns, B-tree/GIN/GiST/BRIN indexes
Code	≈10,500 lines of Python (`src/pgfuzz`), 148 unit tests passing
Grammar validity, meeting 1	83.4% → 83.1% → 88.4% → 100% (966/966) over four tuning rounds
Grammar validity now	99.5–99.7% of statements run without error on master
Throughput (one worker)	≈17–23 grammar test cases per second with all oracles on (3,166 in 180 s full system; 5,516 in 240 s plain grammar)
Plan variants	18 planner configurations (Section 8.5)
Metamorphic check kinds	12 (TLP-WHERE, TLP-HAVING, TLP-aggregate, NoREC, expression, predicate, NOT NULL, ORDER BY drop, CTE, table fence, RLS, CERT)
Real bugs found	4 previously unknown (BUG-01, BUG-02, #19802, #19803); two confirmed by committers, #19803 fixed on master
Benchmark	23 reverted fixes (31 Jul–16 Oct 2026); 20 observable: 8 wrong results, 5 internal errors, 4 assertions, 2 missing errors, 1 wrong error
Crash-only visibility	9 of 20 observable benchmark bugs
RQ3 smoke (15 min/arm)	Grammar+Oracles and +LLM found `rb-dfcf6fa7` at ≈160 s; Random and plain Grammar found none
Precision ledger	93 candidates: 7 real, 15 duplicates, 37 false positives (10 classes, all fixed), 28 flaky, 6 open; precision 0.16 (0.37 with duplicates)

What	Value (source)
Tree reduction	116 → **79** tokens on a real BUG-01 case
Data profile	predicates selecting 5–95% of rows: 29.3% → **58.6%**
Bandit weights	419 → 533 distinct plan shapes per 1,000 test cases, same validity
New rewrites	798 live checks of the new kinds, 0 false positives
RLS / CERT	532 / 58 live checks, 0 false positives
LLM	Gemini Flash (no Claude key); 10 commits, 4 accepted, 25 rounds, 0 USD; +5 features, no coverage gain
Experiment design	RQ1: 7 arms × 5 trials × 1 h; RQ3: 4 arms × 5 trials × 1 h; 4 workers in parallel
Statistics	medians, IQR, 95% bootstrap CI, Mann–Whitney U, Vargha–Delaney $\hat{A}_{12}$

3 Team and grading

Venu – recent PostgreSQL features, campaigns and new grammar rules. **Bhuvan** – features and first pass on every bug candidate (reproduce, shrink, decide). **Gowtham** – experiment, statistics, LLM arm, paper. All three must be able to explain the grammar, the oracles, the statistics and the plan; this document is the script.

Component	Weight	How this project addresses it
Implementation and results	50%	Working three-arm fuzzer, 12 metamorphic oracles plus plan-variant and cross-version oracles, triage, LLM arm, benchmark, campaigns. Top marks need an upstream-accepted bug: BUG-01 and BUG-02 have patches ready to file.
Meetings and presentation	25%	This guide, the slides, live demo (Section 17).
Paper (ACM/FSE)	25%	Five graded parts: motivating example (BUG-02, real), workflow (figure + Steps 1–5), RQs (four), related work (17 verified references), threats.

4 What happened, in order

This section is the history of the project. It explains why the system looks the way it does: most design decisions were forced by something that went wrong on a real server.

4.1 Before meeting 1 (until 16 October)

Stage 1: the harness. An executor (`src/pgfuzz/reference/pg_executor.py`) runs a test case against a real PostgreSQL server, one savepoint per statement, rolled back at the end; it reads the server log for `TRAP` (assertion), `PANIC` and "terminated by signal" lines; it recovers after crashes. An oracle (`src/pgfuzz/eval/oracle.py`) classifies each test case (crash, assertion, internal error `XX000`, timeout, ...). A first wrong-result oracle re-ran each deterministic query under 17 planner settings (the *plan-variant* oracle). Every trial writes a resumable run log (`events.jsonl`, `timeline.jsonl`, `summary.json`) and a report builder produces tables, plots and statistics.

Stage 1: the random/mutation arm (`src/generator/`). Seeds are built with a small SQL AST; they are then mutated as *text* (flip a character, delete a span, swap an operator, splice a keyword). The mutators know nothing about types or scopes, so many mutants are invalid. It ignores feedback on purpose: it is the unguided baseline.

Stage 2: the typed grammar. We designed our own grammar format (`.pgg`) and a derivation engine with scopes and type variables (Section 7). Four tuning rounds of 300 test cases each on the same server and seed raised server-side validity:

Round	Statements	Accepted	Semantic err.	Runtime err.	Timeouts	Time
1	812	677 (83.4%)	76	57	0	16.7 s
2	828	688 (83.1%)	39	89	7	82.9 s
3	929	821 (88.4%)	28	74	6	56.1 s
4	966	966 (100%)	0	0	0	22.1 s

Round 2 looks like no progress (83.1%), but the mix changed: semantic errors halved, more statements reached execution, and one rule (`generate_series` up to a table column) wasted the clock with million-row series. The edits were always driven by the error list, never at random. This is the number we showed at meeting 1. It is a validity number, not a bug.

4.2 Meeting 1 (16 October)

We presented the three claims, the architecture, the RQs, and a plan with seven priorities: (1) long grammar campaigns, (2) metamorphic oracles on derivation trees, (3) cross-version testing, (4) triage automation, (5) the commit-directed LLM loop with diff coverage, (6) a re-injected bug benchmark with statistics, (7) stretch goals (plan-shape feedback, a CERT-style performance oracle, a row-level security oracle).

4.3 The bug hunt (16–17 October)

Setup. In a cloud container we built PostgreSQL master at `26bd08fb` with assertions and PostgreSQL 18.6 (`REL_18_STABLE`) as a reference. We ran the grammar campaign (`configs/cloud_hunt.yaml`, two trials of six hours) and two cross-version join fuzzers (`src/xversion/joinfuzz*.py`) that run the same query on master and on 18.6 with random planner settings.

Candidates.

1. A plan-variant wrong result under `enable_hashagg=off`: *false positive*. `ARRAY(SELECT ... EXCEPT ALL ...)` has no `ORDER BY`, so the array's element order legitimately depends on the plan. Fixed in the determinism filter `eligible()`.
2. An assertion failure about 600 s into trial 0 (test case 13306): `TRAP: failed Assert("list_length(rcexpr->inputcollids) == nopers")`, `execExpr.c:2078`. **A real bug (BUG-01)**, see Section 14.2.
3. More `ARRAY(SELECT)` false positives from a trial that started before the fix.
4. A crash-recovery failure on one server after the BUG-01 crash chain: `FATAL: could not create file "base/5/165792": File exists`. We traced it in the WAL to an unlogged sequence created in an open transaction, read the recovery code (`ResetUnloggedRelations`), but could not reproduce it from a clean data directory. Unreproduced bugs are not reported. Evidence is kept in `docs/bughunt/evidence/`.

Lessons that became code. The failure writer once saved the wrong test case for a signature; the determinism filter needed the `ARRAY(SELECT)` rule; a cross-version mismatch came from `DEFAULT now()` in our own seed data (the clock differs between servers) – fixed with deterministic data and by excluding clock functions.

4.4 The build session (23 October)

This is where most of the meeting-2 system was built, in this order.

1. **Typed node spans.** The engine wraps each node's tokens in invisible markers, so after rendering it knows exactly which tokens each tree node produced. This is what lets an oracle edit one node.
2. **Metamorphic oracles on derivation trees** (TLP, NoREC, expression rewrites, CTE toggle, table fence). In the very first sample of 150 test cases, the scalar-subquery rewrite `e` → `(SELECT e)` found **BUG-02** (Section 14.3). A 600-test-case live check then gave 907 checks, 0 false positives.
3. **Plan-shape guidance and metric.** A test case whose plan has a new shape (digest of node types in `EXPLAIN`) enters the corpus.
4. **Cross-version executor** (master vs 18.6) with a sequence prelude, deterministic data, clock functions excluded: 1,155 comparisons, 0 mismatches after the fixes.
5. **Triage pipeline**: reproduce, reduce (statements, clauses, tokens; pairs for metamorphic checks), report, bisect script, precision ledger. On BUG-02's candidate: 4 statements / 107 tokens → 1 statement / 36 tokens in 307 runs.
6. **LLM arm**: commit miner, cached prompts, client with manual mode, validation gate, diff coverage, repair loop, ledger.
7. **RQ3 benchmark**: 23 fixes reverted. The first PoC verification showed only 12/23 observable. Two of our own mistakes caused most of the gap: the PoC tables were called `i4`/`i8`, names our schema already uses for *indexes*, so the PoCs failed at `CREATE TABLE`; and the digest masked float aggregates so completely that `0` vs `NaN` looked equal. After renaming (prefix `rb_`), keeping NaN/Infinity in masked floats, adding `ANALYZE`, and fixing two PoCs: **20 of 23 observable**, the same symptom with all 23 injected together (no masking).
8. **Smoke run 1** (4 min per arm on the injected server): the grammar arm **rediscovered BUG-01**; five metamorphic candidates turned out to be **our** false positives: TLP-HAVING removed `HAVING` from a level without `GROUP BY` (turning a one-group aggregate into a plain query), and `DISTINCT` over `numeric` may keep `21.0000` or `21.00` depending on the plan. Both fixed with tests. Also: all four workers shared one reference server and raced on schema setup; fixed with one database per worker.
9. **Smoke run 2** (15 min per arm): Grammar+Oracles and +LLM found re-injected bug `rb-dfcf6fa7` at about 160 s; attribution by replay on the solo servers picked the right one of two bugs that share an error message. Two random-arm "finds" were **our** fault: a PoC replay we ran during the campaign crashed the shared server and the harness blamed the random arm's statement. Fixed: crash evidence now must carry the test case's own backend PID. A cross-version difference (`CORR` of constant input: NULL on master, 1 on 18.6) is an intended master change; those functions are excluded.
10. **Paper draft, execution plan, progress report.**

4.5 The plan-coverage round (24 October)

We re-read the meeting-1 plan and slides line by line and implemented every item still missing: TLP for aggregates and the remaining rewrites of plan Section 3.4; grammar-tree reduction ("one tree, four jobs"); localization of wrong results (plan oracle × rewrite oracle); feedback-weighted grammar alternatives (bandit); a data profile for boundary and selective literals; the RLS leak oracle; the CERT oracle; LLM-proposed equivalence rewrites with a value probe; the eager-aggregation plan variant; upstream bug-report drafts; 95% bootstrap confidence intervals. Each was checked on master (Section 14.7). The traceability matrix is `docs/PLAN_COVERAGE.md`.

Things that went wrong and what they taught us

- A richer grammar exposes more corners where two "equivalent" queries are not equivalent; oracle precision, not generator power, limits a wrong-result fuzzer. Every false-positive class became a fix plus a test.
- Shared servers are dangerous: our own manual replay corrupted a campaign's attribution; the fix is PID-checked evidence.
- Our own PoCs were wrong twice (name clash, stats-dependent plans); ground truth needs verification too (`bench verify`).
- Sampling is not proof: a false LLM rewrite passed 109 sampled checks and was caught only by evaluating it on NULL.
- `pkill -f pattern` from a shell whose command line contains the pattern kills that shell; wait loops that `pgrep` their own pattern never end. Small, but they cost us runs.

Part II
Background

5 Concepts we rely on

5.1 How PostgreSQL runs a query

A SQL string goes through five stages: **parser** (text → parse tree; syntax errors, SQLSTATE class 42601), **analyzer** (names, types, collations; semantic errors such as unknown column or operator, class 42), **rewriter** (views, rules, row-level security quals), **planner/optimizer** (chooses join order, join methods, scans, aggregation strategy; folds constants; calls *support functions* that may simplify function calls; removes useless joins; pushes predicates down), **executor** (runs the plan tree node by node). Most wrong-result bugs we care about live in the planner: a transformation that is correct only under an assumption ("this column cannot be NULL", "this join cannot duplicate rows") that does not hold.

Planner settings (GUCs). A session can switch planner methods off, for example `SET enable_hashjoin = off`. These are hints, not semantics: every setting must give the same rows. This is what the plan-variant oracle exploits.

Assertion builds. PostgreSQL compiled with `-enable-cassert` checks internal invariants with `Assert()`. A failure aborts the backend and logs `TRAP: failed Assert("..."), File: "...", Line: ...`. Release builds skip the checks, so the same root cause often shows up there as a silent wrong result – exactly what BUG-01 did on 18.6.

SQLSTATE classes. `XX000` (internal error, "should not happen") is almost always a bug. 42xxx (syntax/semantic) and 22xxx (data, for example division by zero) are normal for fuzzed SQL.

Three-valued logic. A predicate is TRUE, FALSE or NULL (unknown). `WHERE` keeps only TRUE rows. `NOT NULL` is NULL. This is why TLP partitions into p, `NOT p` and `p IS NULL`, and why `a < b` equals `NOT (a >= b)` exactly (both are NULL when an input is NULL).

Row-level security (RLS). A table can have policies (`CREATE POLICY ... USING (pred)`); a non-owner, non-superuser role sees only rows where the predicate is TRUE. Superusers bypass RLS.

Cardinality estimation. The planner estimates the number of rows each plan node returns (`EXPLAIN` shows "rows="). Bad estimates cause bad plans (performance bugs), not wrong results.

5.2 The test-oracle problem and how we answer it

A fuzzer needs an *oracle* that decides whether an output is wrong. Crashes and assertion failures are their own oracle. Wrong results need one of:

Differential testing. Compare two implementations (RAGS, Slutz VLDB 1998). We compare one server with itself under other planner settings (*plan variants*) and with the last release (*cross-version*).

Metamorphic testing. Transform the query into another query whose result must be related in a known way (usually: equal multiset). Known instances: **TLP** (Rigger & Su, OOPSLA 2020): $Q = Q_p \uplus Q_{\neg p} \uplus Q_{p \text{ IS NULL}}$; **NoREC** (Rigger & Su, ESEC/FSE 2020): `count(*) WHERE p` equals the number of rows where p is TRUE when p is evaluated in the select list (where the optimizer cannot use it); **EET** (Jiang & Su, OSDI 2024): equivalent expression transformations; **PQS** (OSDI 2020): a chosen "pivot" row must be in the result.

SQLancer implements TLP/NoREC/PQS on strings produced by its own small generators. Our contribution is doing these on *typed derivation trees* of a much larger grammar, at any query level, plus RLS and CERT checks.

5.3 Fuzzing background

Mutation-based fuzzing (AFL-style) mutates inputs blindly; for SQL most mutants are syntax errors. Our random arm is this baseline.

Grammar-based fuzzing derives inputs from a grammar, so they are syntactically valid. A *typed* grammar also makes them type-correct. A **derivation tree** records which alternative each rule chose; the leaves read left to right are the SQL text.

Coverage-guided / greybox fuzzing keeps inputs that cover new code (SQUIRREL, CCS 2020; SQLRight, USENIX Security 2022). We use cheaper signals: new SQL features, new feature pairs, new plan shapes (Query Plan Guidance, Ba & Rigger, ICSE 2023) and new bug signatures.

Directed fuzzing (AFLGo, CCS 2017) steers toward chosen code. We steer toward recent commits through the LLM and measure it by *diff coverage*: changed lines executed / changed executable lines.

LLM-based fuzzing (Fuzz4All, ICSE 2024) uses the model as the input generator. We keep the LLM off the hot path: a few calls per commit, then the grammar generates thousands of variations.

Multi-armed bandit / multiplicative weights. An online learning rule that increases the weight of choices that paid off; we use it for grammar alternatives.

5.4 Reduction, benchmarks, statistics

Delta debugging (ddmin, Zeller & Hildebrandt, TSE 2002) removes chunks of a failing input while the failure remains.

Hierarchical delta debugging (HDD, Misherghi & Su, ICSE 2006) does it level by level over a tree (we use the parenthesis tree of the SQL).

Magma (Hazimeh et al., 2020) is a ground-truth fuzzing benchmark of real bugs ported into programs. Our RQ3 benchmark is the same idea built by *reverting* fixes (mutation testing, L13: inject faults, measure detection).

Evaluating fuzzers (Klees et al., CCS 2018): repeated trials, same budget, medians, statistical tests; one run proves nothing.

Mann–Whitney U is a non-parametric test of whether one arm's values tend to be larger; **Vargha–Delaney** $\hat{A}_{12}$ is the probability that a random trial of arm B beats one of arm A (0.5 = no difference; ≥ 0.71 large). Arcuri & Briand (ICSE 2011) recommend both for randomized algorithms. A **bootstrap confidence interval** resamples the trials with replacement to estimate the uncertainty of the median.

gcov counts executions per source line when PostgreSQL is built with `-enable-coverage`.

5.5 Course lectures used

Lecture topic	Where it appears
L2, L4, L5: hybrid analysis, soundness, precision/recall	commit mining directs a dynamic campaign; generator validity and oracle precision measured (Section 10)
Hoare logic / assertions	each metamorphic rewrite is an assertion: original and rewritten query return the same multiset
L3, L21–22: secure coding, information flow	RLS leak oracle: a restricted role must never see a row its policy hides
L6–L8: AST, CFG, IR	derivation trees as ASTs; gcov line coverage over the CFG
L13: coverage, mutation testing	coverage metrics; the re-injected bug benchmark
L14–L16: black/grey-box and grammar fuzzing	the three arms; feature/plan-shape feedback; bandit weights
L19–20: interval analysis	boundary literals from the observed interval of each column (min±1, type edges)
L27–28: constraint-based analysis	data-aware literals chosen to give a predicate 10–90% selectivity
L29–30: type systems	typed grammar: generation is proof search for $\Gamma \vdash e : T$

Part III
The system

6 Architecture

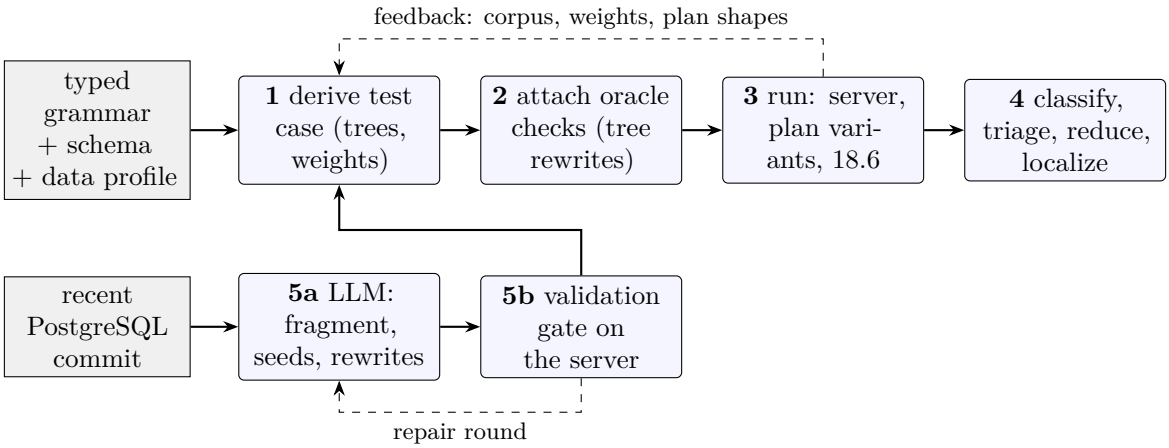


Figure 1: The fuzzing loop (top) and the LLM loop (bottom). Steps match Sections 7–11.

6.1 Repository map

Path	Contents
`src/grammar/postgres.pgg`	the typed grammar
`src/corpus/schema.sql, schema.yaml`	the 9-relation schema (SQL for the server, YAML for the generator)
`src/corpus/rls.sql, profile.json`	RLS policies; data profile
`src/pgfuzz/grammar/`	DSL parser (`dsl.py`), engine (`engine.py`), generator, rewriter (oracles), data profile, tree I/O
`src/pgfuzz/reference/`	PostgreSQL executor, cross-version executor
`src/pgfuzz/eval/`	oracle, differential digests and determinism filter, signatures, metrics, statistics, features, known-bug catalog
`src/pgfuzz/triage/`	tokenizer, token reduction, tree reduction, localization, triage driver, precision
`src/pgfuzz/llm/`	commit miner, prompts, client, gate, diff coverage, loop
`src/pgfuzz/bench/`	re-injected bug benchmark (verify, attribute)
`src/pgfuzz/runner/, runlog/, report/`	experiment runner, run logs, reports
`src/generator/`	the random/mutation arm (Stage 1)
`src/bench/reinjected/`	benchmark: 23 commits, PoCs, `bugs.yaml`, `verify.json`
`src/configs/`	`rq1.yaml`, `rq3.yaml`, hunt and local configs
`scripts/`	server builds and starts (`build_reinjected.sh`, `rq1_servers.sh`, `rq3_servers.sh`, `ref_server.sh`), bisection, setup
`triage/verdicts.yaml`	the verdict of every triaged candidate (feeds RQ2)
`docs/`	guides, plans, evidence, bug reports (`docs/bughunt/`), paper draft PDF
`paper/`	the ACM/FSE paper (LaTeX)

6.2 One trial

`python -m pgfuzz run configs/rq1.yaml -j 4` plans the trials (arm × trial), and each worker runs one trial against its own server (port = base port + worker). A trial loops: ask the generator for a test case, execute it, classify the result, update metrics and the run log, save a failure if the signature is new, give feedback to the generator. Per trial it writes `manifest.json`, `events.jsonl`, `timeline.jsonl` (snapshots every 30 s), `failures/<signature>/` and `summary.json`. Trials are resumable: re-running the command skips finished trials. The seed of trial k is the same in every arm (paired design).

7 Step 1: the typed grammar and the generator

7.1 The `.pgg` format

- `%type int "int" num ord key ... w=6` declares a SQL type, its classes (`num`, `ord`, `key`, ...) and a weight.
- A rule has alternatives with weights `<4>`; terminals are keywords or quoted strings; `x?0.3` is optional with probability 0.3; `x{1,4/","}` repeats 1–4 times separated by commas.
- Rules can be typed: `expr<T>`, `expr<int>`, `expr<X:key>` (a type variable restricted to a class).

- Attributes: `@scope` opens a new name scope, `@set(agg_ok)` sets a context flag (aggregates allowed), `^` marks elements derived early (FROM before SELECT, so the select list can see the FROM's columns).
- Builtins (Python): `@col(T)` a visible column of type T, `@lit(T)` a literal, `@dlit(T)` a literal from the data of the compared column, `@table` a relation with a fresh alias, `@items` the select list, `@group_keys` makes the level grouped, `@sorted_agg` order-sensitive aggregates made deterministic, and about 20 more for DDL and DML.

A real excerpt:

```
query @scope
  : <8> select_core order_by?0.3 limit_clause?0.06
  | <2> with_clause select_core order_by?0.2
  | <2> set_query
  ...
select_core
  : <6> SELECT distinct?0.12 @items ^from_clause ^where_clause?0.6
  | <3> SELECT @items ^from_clause ^where_clause?0.5 GROUP BY ^@group_keys having_clause
    ?0.4
  | <2> SELECT @items ^from_clause ^where_clause?0.5 ^@global_agg having_clause?0.2
  ...
expr<bool>
  : <4> "(" expr<X:key> cmp expr<X> ")"
  | <1.2> "(" @col(X:ord) cmp @dlit(X) ")"
  | <2> "(" expr<bool> AND expr<bool> ")"
  | <1> "(" expr<X:key> NOT?0.3 IN "(" expr<X>{1,4/","} ")" ")"
  ...
```

Conventions that make oracles possible: every binary operator is parenthesised (precedence never changes types); every column is qualified with a unique alias; order-sensitive aggregates order by their own argument.

7.2 The engine

The engine expands a rule into a tree, choosing an alternative by weight, binding type variables (`X`) consistently, tracking scopes (which aliases and columns are visible, correlated or isolated) and flags (grouped level, inside an aggregate). It backtracks when an alternative cannot be completed (`Fail`) and prefers shallow alternatives past a soft depth. Each alternative knows its minimal derivation depth (`mind`). While rendering, each node's tokens are wrapped in invisible markers (`\x01<nid>` … `\x02<nid>`), so the exact token span of every node is known afterwards. A `MINIMAL` marker on a guide node makes the engine derive that subtree with its smallest alternatives (used by tree reduction).

Replay. Given a guide tree, the engine follows its choices (same alternatives, same columns, same literals), re-binding names and types in the current schema state. Mutation and reduction both use replay, so their results stay valid. Trees are stored with failures as JSON using stable alternative *ordinals* (uids are process-local) and exact container types; 599 of 600 test cases replay to identical SQL in a fresh process.

7.3 The generator

A test case is 0–2 setup statements (DDL such as `CREATE INDEX/TABLE`, DML, `ANALYZE`, `SET LOCAL` of a planner setting) followed by 1–2 queries or DML statements.

Fresh or mutated. With probability 0.5 a corpus entry is picked (weighted by its finds and how often it was picked) and mutated: *regen* (re-derive one subtree), *splice* (graft a subtree from another corpus entry), *insert* (add a statement), *drop* (remove one).

Corpus admission. A test case enters the corpus if it produced a new bug signature, a new SQL feature (catalogue of 148), a new pair of features, or a new plan shape (digest of node types of `EXPLAIN (FORMAT JSON, COSTS OFF)`).

Feedback-weighted alternatives (plan 5.3). Each alternative has a weight multiplier. After every test case, each alternative used in a tree gets reward $r = 1$ if the test case entered the corpus, -0.5 if that tree's statement failed, else 0; its weight is multiplied by $e^{\eta(r-\bar{r})}$ with $\eta = 0.1$ and $\bar{r}$ a running average, clipped to $[0.2, 5]$. *Measured:* 533 vs 419 distinct plan shapes in 1,000 test cases at the same validity; the bandit learned to damp window-function alternatives that often fail. RQ1 contains an ablation arm without it.

Data profile (plan 6, interval and constraint-based analysis). `pgfuzz profile` asks the server for min, max and the 10/25/50/75/90% quantiles of every column (`corpus/profile.json`). With probability 0.3 a literal comes from it: boundary values (observed min±1, max±1, 0, −0, NaN, ±Infinity, the largest int, the empty array) or quantiles. The new builtin `@dlit(T)` takes a quantile of the *column the predicate compares* (the engine remembers the last `@col`), so `col < @dlit` selects a real share of rows. *Measured:* column-vs-constant predicates selecting 5–95% of their table: 29.3% → 58.6%.

Validity tuning loop. `pgfuzz sample -execute` prints the valid rate, the top error messages and the worst alternatives (by valid rate); each fix is to one rule. This loop raised validity from 83% to 100% before meeting 1.

7.4 The schema

Nine relations shared by both arms: `t0` (int, text, float8), `t1` (int PK, int, jsonb, int4range), `t2` (identity bigint, numeric, timestamptz, text[]), `t3` (int NOT NULL, bool, date, interval, int[], text), `t4` (virtual and stored generated columns), `tp` (range-partitioned), `tl` (list-partitioned), `ti` (inheritance parent with child), `v0` (a view). Data is deterministic (no `now()`), with NULLs in most columns. Indexes: B-tree (including multi-column `t3(c0,c2)` for skip scan), GIN, GiST, BRIN, and an expression index with a predicate.

8 Steps 2–3: executing a test case and the oracles

A fuzzer is only as good as its oracle. This section explains every way the system decides that PostgreSQL did something wrong, and, just as important, every rule that stops it from calling correct behaviour a bug.

8.1 The executor

`src/pgfuzz/reference/pg_executor.py` (741 lines) runs one test case on a real server.

1. **Isolation.** The test case runs inside one transaction; each statement in its own savepoint (`SAVEPOINT`, then `RELEASE` or `ROLLBACK TO`). At the end everything is rolled back, so the database is identical for the next test case. Statements that cannot run in a transaction (`VACUUM`, `CREATE INDEX CONCURRENTLY`) use `isolation: autocommit` plus a `reset_sql` script.

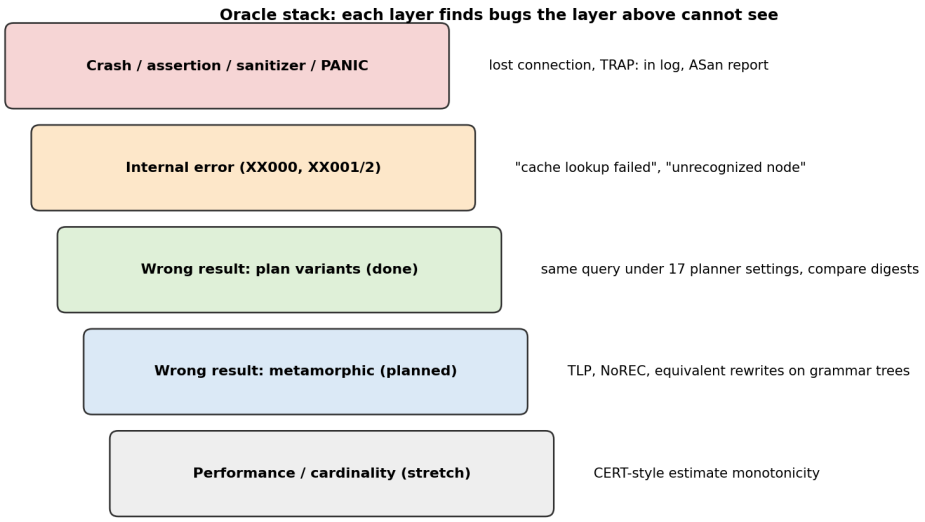


Figure 2: The oracle stack: crash and error oracles see every statement; plan-variant, metamorphic and cross-version oracles see deterministic queries.

2. **Timeout** per statement (`statement_timeout`, default 3 s in campaigns) so a slow plan cannot stall a worker.
3. **Server log.** After each statement the executor reads new lines of the server log and looks for `TRAP:` (assertion), `PANIC:`, "terminated by signal", sanitizer reports and `BACKTRACE:` lines (PostgreSQL prints a C backtrace for internal errors when `backtrace_on_internal_error = on`).
4. **Crash recovery.** If the connection is lost, it waits for the postmaster to finish crash recovery (SQLSTATE 57P03 "in recovery"), reconnects, and recreates the per-worker database if needed.
5. **Digests.** For each successful query it computes an order-insensitive digest of the result (below) so it can compare results without storing them.
6. **Oracle work** for each successful, deterministic query: plan variants, metamorphic checks, plan shape (`EXPLAIN (FORMAT JSON, COSTS OFF)` digest), optionally coverage sampling.

Foreign-crash filter. When one backend crashes, PostgreSQL restarts *all* backends; every other session sees "terminating connection because of crash of another server process" (57P02). Each worker has its own server, but our own manual PoC replay once crashed a server that a campaign was using and the random arm was blamed. Now the executor records its backend PID (`pg_backend_pid()`); crash evidence in the log must name that PID (or a parallel worker of our own query), otherwise the event is classified as *foreign* and ignored (`_foreign_crash`).

8.2 Classification

`src/pgfuzz/eval/oracle.py` turns executor facts into one `Outcome`, in priority order:

1. log evidence: assertion, sanitizer, signal (crash), `PANIC`;
2. connection loss without an orderly SQLSTATE: crash;
3. the SQLSTATE of a failing statement, by class: 42601 → syntax; 42, 0A, 25, … → semantic; 22, 23 … → runtime; 57014 → timeout; 53, 54 → resource limit; **XX000 → internal error**; XX001/XX002 → data corruption. A short list of XX000 messages that are really resource exhaustion ("out of memory" in a hash table, etc.) is demoted;

4. plan-variant facts: a variant that raised an internal error, or returned a different digest → `wrong_result` (variant name recorded);
5. metamorphic facts: a *confirmed* mismatch → `wrong_result` with variant `mm:<kind>`; a CERT violation → `performance` (not a wrong answer).

The outcomes are ordered by severity (ok < syntax < semantic < runtime < timeout < resource < terminated < performance < wrong result < internal error < corruption < panic < assertion < sanitizer < crash). The test case gets the most severe statement outcome; the index of that statement is the *culprit*. Bug candidates are: crash, assertion, sanitizer, panic, corruption, internal error, wrong result, and (reported separately) performance.

8.3 Determinism: which queries may be compared at all

Comparing two runs of a query only makes sense if the correct answer is a single multiset. `eligible(sql)` in `src/pgfuzz/eval/differential.py` accepts a statement only if:

- it is `SELECT`, `VALUES` or `TABLE` and contains no `INSERT/UPDATE/DELETE/MERGE/INTO` (no side effects);
- no `LIMIT`, `OFFSET`, `FETCH`, `TABLESAMPLE`, `DISTINCT ON`, `FOR UPDATE/SHARE`, `SKIP LOCKED` (all choose *some* rows, and which ones depends on the plan);
- no volatile function (`random`, `clock_timestamp`, `nextval`, `gen_random_uuid`, `pg_sleep`, ..., 42 names);
- no order-dependent window function (`row_number`, `ntile`, `lag/lead`, `first/last/nth_value`) and no `ROWS` frame (ties are broken by the plan); `rank`, `dense_rank`, `percent_rank`, `cume_dist` are tie-safe and allowed;
- no order-sensitive aggregate without its own `ORDER BY`: `array_agg`, `string_agg`, `json(b)_agg`, `json(b)_object_agg`, `xmlagg`, `json_arrayagg`, `json_objectagg`, `any_value`, ...;
- no `ARRAY(SELECT ...)` without `ORDER BY` inside (added after candidate 1 of the bug hunt, a false positive).

The grammar helps: `@sorted_agg` writes `array_agg(x ORDER BY x)`, so most aggregates the generator emits *are* eligible.

8.4 Result digests

All values are fetched as text (PostgreSQL's own output format) and canonicalised:

- **numeric scale:** `2.0`, `2.00` → `2` (equal values; `DISTINCT/GROUP BY/UNION` may legitimately keep either representative – found as a false positive in smoke run 1);
- **float −0:** −0 and 0 compare equal, so either may survive;
- **float masking:** if the query has a floating-point aggregate or window (`sum`, `avg`, `stddev`, `corr`, `regr_*`, `percentile_cont`, `OVER`), float columns are reduced to a class: `NaN`, `Infinity`, `-Infinity` or "finite". Summation order changes the last digits legitimately; it cannot turn a finite value into NaN. (Masking *everything* at first hid a re-injected bug: 0 vs NaN. That is why the class is kept.)

Rows are joined with separator bytes, sorted (multiset semantics), and hashed with BLAKE2b (96 bits). Results above 10,000 rows are not compared.

8.5 Oracle 1: plan variants (differential testing inside one server)

The same query is re-run with `set_config(name, value, true)` (local to a savepoint, rolled back) under planner configurations. The settings change *how* PostgreSQL runs the query, never *what* it returns, so any digest difference is a bug (given eligibility). The 18 variants:

Variant	Settings (what it forces)
`nestloop_only`	hash and merge join off
`hashjoin_only`	nested loop and merge join off
`mergejoin_only`	nested loop and hash join off
`no_hashagg`	sort-based grouping
`no_sort_opt`	no incremental sort, no presorted aggregate
`index_scans`	seq scan and bitmap scan off
`seq_scans`	index, index-only and bitmap scans off
`no_memoize_material`	no Memoize, no Material nodes
`parallel`	`debug_parallel_query=on`, zero parallel costs, 4 workers, leader not participating
`partitionwise`	partition-wise join and aggregate on
`no_partition_pruning`	partition pruning off
`geqo`	genetic optimizer from 2 relations
`no_join_reorder`	`join/from_collapse_limit = 1` (syntactic join order)
`tiny_work_mem`	`work_mem=64kB`: hash joins batch, sorts spill to disk
`no_reordering`	no GROUP BY / DISTINCT key reordering (PG 17/18 features)
`no_self_join_elim`	self-join elimination off (new in PG 18)
`no_eager_agg`	eager aggregation off (new on master, Oct 2026)
`jit`	JIT compile everything (costs 0)

Settings unknown to the server are dropped at start (each variant is probed once), so the same config works on 17, 18.6 and master. To keep throughput, each query runs under `per_query = 2` variants chosen deterministically from a hash of the SQL text (`pick_variants`): a failure replays with the same variant, and over a campaign every variant is used. A failure is *materialised* for reports: the culprit statement is preceded by explicit `SET` commands of the variant.

Why this finds bugs. Each variant forces the planner down a different code path (another join method, another aggregation strategy, a parallel plan, spilling). Many optimizer bugs live in exactly one path. Re-injected bug `rb-dfcf6fa7` was found this way in smoke run 2.

8.6 Oracle 2: metamorphic rewrites of the derivation tree

`src/pgfuzz/grammar/rewrite.py` (656 lines). For each eligible query, the rewriter looks at its *derivation tree* and the token span of each node, and builds up to `per_query` checks. A check has a left side (the original query, or a variant of it) and a right side (the rewritten query); both must return the same multiset. Kinds are sampled by weight (partition checks apply to fewer queries, so they get larger weights: TLP-WHERE 6, TLP-HAVING 5, TLP-agg 5, NoREC 3, expression 3, predicate 3, RLS 2, NOT NULL 1.5, CTE 1.5, fence 1.5, ORDER BY 1, CERT 1).

Why do this on a tree and not on a string?

SQLancer builds its own small queries so it knows their structure. We have arbitrary, deeply nested queries from a 421-alternative grammar. The tree tells us exactly which tokens are the `WHERE` predicate of *this* `SELECT` (not of a subquery), which expression has which type, which level is grouped, which columns are in scope, and whether a node sits under an outer join. A string rewrite would have to re-parse SQL and would get all of these wrong some of the time; each mistake is a false positive.

Where a rewrite may apply. Partition checks (TLP) apply to any `select_core` whose result is used as a multiset by the enclosing query: the query itself, a subquery in `FROM`, a set-operation arm, an `IN`/`EXISTS` subquery, a `LATERAL` subquery (`MULTISET_CONTEXTS`). Replacing a multiset by an equal multiset cannot change the statement. A scalar subquery is *not* such a context (it must return one row), so it is not partitioned. Levels with window functions are skipped (a window sees the whole partition; splitting the rows changes it).

The twelve kinds

Kind	Rewrite (left ≡ right)	Soundness conditions / targets
`tlp_where`	`SELECT .. FROM f` ≡ `(.. WHERE p) UNION ALL (.. WHERE NOT p) UNION ALL (.. WHERE p IS NULL)`	Ternary logic partition (Rigger & Su 2020). Not grouped. With `DISTINCT`: `UNION` instead, and only if all output types have a unique text form. Nested arms parenthesised so `INTERSECT` cannot regroup them.
`tlp_having`	`.. GROUP BY k HAVING TRUE` ≡ union of `HAVING p`, `NOT p`, `p IS NULL`	Grouped levels; group keys must have unique text form. Left side keeps `HAVING TRUE`: removing `HAVING` from a level without `GROUP BY` changes it from a one-group aggregate to a plain query (our false positive in smoke run 1).
`tlp_agg`	`SELECT MIN(e) AS c0, COUNT(x) AS c1 FROM f` ≡ `SELECT MIN(t.c0), SUM(t.c1) FROM (three partitions) t`	TLP for aggregates. Only combinable aggregates: MIN, MAX, SUM, COUNT→SUM, BIT_AND/OR/XOR, BOOL_AND/OR, EVERY; no DISTINCT, FILTER or ORDER BY inside; every item exactly one call.
`norec`	`SELECT COUNT(*) FROM f WHERE p` ≡ `SELECT COALESCE(SUM(CASE WHEN p THEN 1 ELSE 0 END),0) FROM f`	NoREC (Rigger & Su 2020): the optimizer can use `p` in the first form (index, pushdown, join pruning) but must evaluate it per row in the second.
`expr`	replace one typed expression node e by an equal one: `CASE WHEN TRUE THEN e END`, `COALESCE(e,e)`, `(SELECT e)`, `e+0`, `e*1`, `e||""`, `NOT NOT e`, `e AND TRUE`, `e OR FALSE`, `CASE WHEN e THEN TRUE WHEN NOT e THEN FALSE END`	EET-style (Jiang & Su 2024). Rewrites are chosen *by the node's type* (`+0` on float8 is excluded: $-0+0$ prints 0). No rewrite inside aggregates/windows for `(SELECT e)`. **`scalar_sub` found BUG-02.**
`pred:in_or`	`e IN (a,b)` ≡ `(e=a) OR (e=b)` (also `NOT IN`)	Targets PG 18's OR-to-`= ANY` transformation.
`pred:cmp_negate`	`a < b` ≡ `NOT (a >= b)`	Exact under 3-valued logic for types with a B-tree total order.

Kind	Rewrite (left ≡ right)	Soundness conditions / targets
`pred:between`	`x BETWEEN lo AND hi` ≡ `x >= lo AND x <= hi`	Range/index-condition code.
`notnull`	`WHERE p` ≡ `WHERE (p AND c IS NOT NULL)` for a NOT NULL column c that p already uses	Targets NOT-NULL-based simplification (PG 18). Not under outer joins (NULL-extended columns); not generated columns; c must be in the scope of the predicate, not of a nested sub-select (`_nested_under`; a bug we fixed).
`order_drop`	remove the statement's own `ORDER BY`	Multiset unchanged (no LIMIT, by eligibility). Exercises sort elimination and incremental sort.
`cte_materialize`	`WITH x AS (..)` ↔ `AS MATERIALIZED (..)` ↔ `AS NOT MATERIALIZED`	Inlining vs materialisation of CTEs.
`table_fence`	`t AS a` ≡ `(SELECT * FROM t OFFSET 0) AS a`	`OFFSET 0` is an optimisation fence: no pull-up, no pushdown into the table.
`rls`	see below	information-flow oracle
`cert`	see below	performance oracle

Example (TLP-WHERE), generated and rewritten by the tool:

```
-- original (left side keeps everything except WHERE)
SELECT t0_1.c0 AS c0 FROM t0 AS t0_1
-- right side
(SELECT t0_1.c0 AS c0 FROM t0 AS t0_1 WHERE ((t0_1.c0 > 5)))
UNION ALL
(SELECT t0_1.c0 AS c0 FROM t0 AS t0_1 WHERE NOT ((t0_1.c0 > 5)))
UNION ALL
(SELECT t0_1.c0 AS c0 FROM t0 AS t0_1 WHERE ((t0_1.c0 > 5)) IS NULL)
```

Confirmation. A first mismatch is not enough: both sides are run again. Only if both re-runs return the same digests as the first runs is the mismatch *confirmed*; otherwise it is *unconfirmed* (the query is not deterministic after all) and ignored. Errors on one side ("side errors", for example a rewrite that overflows) are counted, not reported, unless they are internal errors.

LLM-proposed rewrites come in as templates such as `COALESCE({e}, {e})` with a type; they are added to the same `expr` pool after passing the rewrite gate (Section 11.4).

8.7 Oracle 3: the RLS leak oracle (information flow)

Setup. `src/corpus/rls.sql` creates role `pgfuzz_rls` (SELECT on all tables) and one policy on five tables: `t0`: `(c0 % 3) <> 0`; `t1`: `c1 IS NULL OR c1 < 30`; `t3`: `c1 IS NOT FALSE`; `tp`: `c0 > 10`; `ti`: `c1 IS DISTINCT FROM 'a'`. The executor connects as a superuser, which bypasses RLS, so these policies change nothing for the other oracles. As the role, 131 of the 203 rows of `t0` are visible.

The check. Left: the original query run with `SET LOCAL ROLE pgfuzz_rls` (policies apply). Right: the query run as superuser with every protected table reference `t AS a` replaced by `(SELECT * FROM t WHERE (policy)) AS a`. Both must be equal. A left side with *more* rows means a **leak**: the restricted role saw a row the policy hides – for example because a leaky function or an operator

was pushed below the security barrier, or a plan path forgot the policy qual. Fewer rows means a wrong result.

Why it matters. It connects the project to the course's information-flow lectures (L21–22): the policy is a confidentiality label; the oracle checks non-interference on concrete data. RLS bugs are security bugs (PostgreSQL has had CVEs in this area, e.g. CVE-2023-2455 and CVE-2024-10976). *Measured:* 532 live checks on master, 0 false positives.

8.8 Oracle 4: CERT (cardinality-estimation restriction testing)

CERT (Ba & Rigger, ICSE 2024): adding a restriction to a query must not *increase* the planner's row estimate. Left: `EXPLAIN` of the query without its `WHERE`; right: `EXPLAIN` of the query. If the estimate of the restricted query exceeds the unrestricted one, the outcome is `performance` (a bad estimate leads to bad plans, but not to wrong results). Restricted to ungrouped, non-`DISTINCT`, window-free levels, where the monotonicity really holds. *Measured:* 58 checks, 0 violations. We report these separately and do not count them as wrong results.

8.9 Oracle 5: cross-version (master vs 18.6)

`src/pgfuzz/reference/xversion_executor.py`. Every test case runs on the server under test (master, assertions on) and on a reference (18.6 release). For each statement that succeeded on both and is `xversion_eligible`, the digests are compared; a difference is reported as a wrong result with variant `xversion`, through the same path as plan variants.

- **Why:** a new optimisation that is wrong under *every* planner setting (no GUC turns it off) is invisible to plan variants but disagrees with a release that does not have it. Regressions in unreleased code are also what upstream fixes fastest ("open items" before a release).
- **Exclusions:** clock and instance functions (`now`, `current_timestamp`, `version`, `pg_backend_pid`, `current_setting`, ...) and functions whose behaviour changed *on purpose*: `corr`, `regr_r2/slope/intercept` (master commit `649828769` keeps S_{xx} exactly 0 for constant input, so `corr(x,x)` over equal values is NULL on master, 1 on 18). We found that difference in smoke run 2 and confirmed it in the commit log.
- **Sequences** are not transactional: a `prelude_sql` resets identity sequences on both servers before every test case.
- **Per-worker reference databases** `ref_<worker>`, created by the executor (`create_database`): four workers sharing one reference database raced on schema setup in smoke run 1.
- Statements that fail on only one side are counted (`test_only_ok`, `ref_only_ok`: new features, new errors) but not reported.

Measured: 1,155 comparisons, 0 mismatches after the fixes.

8.10 Signatures (deduplication)

Thousands of failing test cases may share one root cause. Each candidate gets a signature (`src/pgfuzz/eval/signature.py`), strongest first:

stack top 5 symbolised frames of the backtrace, generic frames (`abort`, `ExceptionalCondition`, `PostgresMain`, ...) removed;

assert assertion expression + source file;

message SQLSTATE + error message with literals, numbers and hex abstracted (`N`, `?`, `H`);

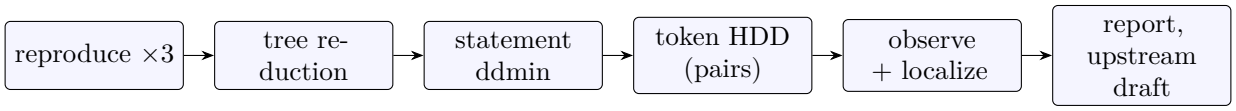
diff for wrong results: oracle/variant + verb and SQL features of the culprit statement;

weak crash without stack: signal + statement shape.

A failure directory is written only for the first test case of each new signature. "Unique bug candidates" in the results are counted by signature, and the strength is logged, because signature quality is a threat to validity.

9 Step 4: triage – from a failure to a bug report

A campaign produces failure directories (one per new signature). Most candidates are not reportable as they are: test cases are long, some are flaky, some are our own false positives. `python -m pgfuzz triage <config> <failure dir>` (`src/pgfuzz/triage/triage.py`) turns a candidate into either a verdict *flaky* or a reduced reproducer with a draft report.



9.1 Reproduce

The candidate is executed three times by the *same executor and oracle* as the campaign, configured for its oracle: only the failing plan variant, or only its metamorphic check (with confirmation on). `same_failure` decides whether a re-run shows *the same* failure, not just any failure:

- assertion: same file and same expression;
- wrong result: same variant / oracle kind;
- internal error, corruption, panic: same normalised message;
- crash without assertion: any crash of the same statement.

$0/3 \Rightarrow$ verdict *flaky*, nothing else is done (2 of our 18 candidates). This criterion is also what every reduction step uses.

9.2 Reduction, three levels

1. Tree reduction (`src/pgfuzz/triage/tree_reduce.py`) – the fourth job of the derivation tree. The failure directory stores the trees of the test case (JSON, stable alternative ordinals). Moves, biggest expected gain first:

1. drop a whole statement;
2. drop one element of a repetition (`#rep`);
3. replace a subtree by the *smallest derivation of the same rule and type*: the node is replaced by a `MINIMAL` marker and the engine derives it with its minimal-depth alternatives.

After each move the generator *replays* the trees (re-binding names, aliases and types), so almost every candidate is valid SQL: the server's time goes to the question that matters (does the failure remain?), not to rejecting broken SQL. A candidate is only tried if it is smaller in tokens; after each success the search restarts from the biggest moves.

2. Statement ddmin (`reduce_statements`): remove setup statements that are not needed (Zeller & Hildebrandt's delta debugging), always keeping the culprit.

3. Token HDD (`TokenReducer`, `reduce.py`): hierarchical delta debugging over the *parenthesis tree* of the culprit statement (Misherghi & Su). Coarse passes first: remove a clause (`WHERE`, `ORDER BY`, ...), remove one list element, unwrap a parenthesised sub-expression or replace it by a small literal (`NULL`, `1`, `TRUE`, `FALSE`, `0`, `''`); then single tokens. Results are cached by text; budget 500 tests or 15 minutes.

Pairs. For a metamorphic candidate (query Q and rewrite Q') both sides must stay equivalent while being reduced. `split_pair` computes the common prefix and suffix of the two token lists and leaves a `HOLE` where they differ; the reducer edits the shared skeleton, and both sides are materialised by filling the hole with each side's middle. So every edit is applied to both sides at once and the hole is never removed.

Measured: tree reduction vs token reduction (BUG-01)

A real grammar test case failing BUG-01's assertion: 3 statements, 590 tokens.

Reduction	Final culprit	Server runs	Time
statements + tokens only (`-no-tree`)	116 tokens	406	33.7 s
tree first, then statements + tokens	**79 tokens**	555	39.6 s

The tree stage alone: 590 → 136 tokens in 150 runs (196 replays). Token reduction gets stuck in local minima (removing any single token breaks the SQL); the tree can jump to a *different, smaller* valid derivation. Evidence: `docs/evidence/tree_reduction/`. BUG-02's first candidate: 4 statements / 107 tokens → 1 statement / 36 tokens in 307 runs (token level, before tree reduction existed).

9.3 Observe and localize

After reduction, triage runs the queries to show (both sides for a pair, or the query under default and failing settings) and records up to 30 result rows on the test server and, if configured, on the 18.6 reference.

Then `src/pgfuzz/triage/localize.py` combines the two oracle families (plan section 0.3 "plan oracle × rewrite oracle"): it re-runs the reduced queries under *every* plan variant and reports, per variant, whether the answer changes and whether the two sides agree. Verdicts:

planner the disagreement disappears when some settings are switched off: those code paths are the first suspects. A table (`SUSPECTS`) maps each setting to source files, e.g. `enable_hashjoin` → `executor/nodeHashjoin.c, optimizer/path/joinpath.c`; `enable_eager_aggregate` → `optimizer/path/allpaths.c, optimizer/util/relnode.c`.

not toggled by planner settings no setting removes it: suspect an always-applied rewrite, constant folding, a planner support function or expression evaluation. **BUG-02 is localized this way**, and its true cause is exactly a planner support function (`rangetypes.c`).

sides agree by default the reduction went wrong (a warning).

9.4 Outputs

Per candidate, in `results/triage/<signature>/`:

- `reduced.sql`: setup statements, then the culprit; for plan variants the `SET` commands are written out; for metamorphic candidates both queries, so the file is a stand-alone reproducer for `psql`;

- `triage.json`: reproduction counts, sizes before/after, runs, seconds, observed rows, localization, server versions;
- `report.md`: a readable report;
- `upstream_report.txt`: a draft for the `pgsql-bugs` form, in the shape PostgreSQL developers expect (version and build, what happens, the minimal script, actual output on master and on the release, the localization, the top 12 backtrace frames, and two placeholders we must fill by hand: *expected behaviour with a justification from the SQL standard or the documentation*, and the bisected commit).

The tool never writes the "expected" part: deciding that something is a bug is a human judgement.

Bisection. `scripts/bisect_pg.sh GOOD BAD reduced.sql MODE` runs `git bisect` on a full-history PostgreSQL clone, building each step into a scratch prefix. Modes: `crash` (backend crash or assertion), `error` (psql reports an ERROR), `output` (differs from an expected output), `differ` (the last two queries of `reduced.sql` return different rows – for metamorphic and plan-variant candidates). Commits that do not build are skipped (exit 125).

10 The precision ledger (RQ2 and claim 2)

Every triaged candidate gets an entry in `triage/verdicts.yaml`: signature, oracle, verdict (*real, duplicate, false_positive, not_a_bug, flaky*), a note, `fixed: true` if the false-positive class has been fixed in the tool, and the upstream link. `python -m pgfuzz precision` computes (`src/pgfuzz/triage/precision.py`):

$$\text{precision} = \frac{\text{real}}{\text{real} + \text{false positive} + \text{not a bug}}, \qquad \text{after fixes} = \frac{\text{real}}{\text{decided} - \text{fixed}}.$$

Duplicates count towards neither side; flaky entries are reported separately.

The ledger today (18 candidates).

Oracle	Candidates	Real	False pos.	Duplicates	Precision	After fixes
assertion	2	1	1	0	0.50	1.00
crash	1	0	1	0	0.00	–
cross-version	3	0	2	0	0.00	–
internal_error	2	1	0	0	1.00	1.00
metamorphic:expr	1	1	0	0	1.00	1.00
metamorphic:tlp_having	4	0	4	0	0.00	–
plan-variant	5	0	5	0	0.00	–
all	18	3	13	0	0.19	1.00

Signature	Oracle	Verdict	Note
`cde9712bc8db`	assertion	real	BUG-01: JSON_TABLE does not assign collations to context item / PASSING (17, 18, master)
`tc112-sample-s`	metamorphic:expr	real	BUG-02: range containment support function returns true/-false for a NULL element (17, 18, master)
`11abedc85d0c`	plan-variant	false_positive	ARRAY(SELECT .. EXCEPT ALL ..) without ORDER BY: element order depends on the plan. Fixed in differential.elig
`67b3ae706302`	plan-variant	false_positive	ARRAY(SELECT .. GROUP BY ..) without ORDER BY (same root cause as 11abedc85d0c; produced by a run started befo
`b73533693016`	plan-variant	false_positive	ARRAY(SELECT .. GROUP BY CUBE ..) without ORDER BY (same root cause)

Signature	Oracle	Verdict	Note
d437283ebe37	plan-variant	false_positive	ARRAY(SELECT .. GROUP BY CUBE ..) inside jsonb_build_object, without ORDER BY (same root cause)
tc150-xversion	cross-version	false_positive	t2.c2 DEFAULT now() differed between servers. Fixed: deterministic seed data, clock functions excluded
0a5d35b1e10a	metamorphic:tlp_having	false_positive	HAVING without GROUP BY makes a one-group aggregate; the unpartitioned side dropped HAVING (0 rows vs 1). Fixe
8ab766603c73	metamorphic:tlp_having	false_positive	same root cause as 0a5d35b1e10a (no GROUP BY at the level)
14fb341a3726	metamorphic:tlp_having	false_positive	same root cause as 0a5d35b1e10a (no GROUP BY at the level)
63e19036ae9f	metamorphic:tlp_having	false_positive	same root cause as 0a5d35b1e10a (no GROUP BY at the level)
aa8a139ce4a2	plan-variant	false_positive	DISTINCT over numeric: 21.0000000000000000 and 21.00000000000000000000 are equal, hash and sort keep different
7ffcd2cb45a7	cross-version	flaky	did not reproduce (0/3) on master vs 18.6; depends on state left by an earlier DML statement of the test case
4fedb431a765	internal_error	real	re-injected benchmark bug rb-dfcf6fa7 (could not find pathkey item to sort); found by Grammar+Oracles and +LLM
6efd24cf879b	assertion	false_positive	harness artifact: our own PoC replay crashed the same server; the TRAP had another backend's PID. Fixed: crash
ca37f4a4a5e1	crash	false_positive	same harness artifact as 6efd24cf879b (connection lost because another backend crashed)
256dc4599fec	cross-version	not_a_bug	CORR(1.0/3, 1.0/3) is NULL on master, 1 on 18.6: intended master change keeping Sxx exactly 0 for constant inp
5f795c1e0a66	internal_error	flaky	XX000 Append child's targetlist doesn't match Append, on the injected server; 0/3 on the injected server, mast

How to read it (be ready for this question).

- The 3 real ones: BUG-01 (assertion), BUG-02 (metamorphic expression rewrite), and the re-injected benchmark bug `rb-dfcf6fa7` (internal error), which counts as real because it is a real historical PostgreSQL bug, put back on purpose.
- 13 false positives / not-a-bug come from **6 classes**, all fixed in the tool with a test: (1) `ARRAY(SELECT)` without `ORDER BY` (5 plan-variant candidates); (2) clock-dependent seed data across versions; (3) TLP-HAVING without `GROUP BY` (4); (4) `DISTINCT` over numeric scale; (5) the shared-server PoC replay (2, assertion + crash); (6) the intended `corr` change (not a bug).
- Raw precision 0.19 is low *on purpose*: we log every false positive we ever produced, including during development. "After fixes" 1.00 means every remaining decided report of a class we have not fixed was real. The paper reports both and the per-oracle counts of the long campaigns, which will be the honest number.
- The 798 new-kind checks, the 532 RLS checks, the 58 CERT checks and the 1,991 metamorphic checks of the full-system smoke produced 0 new false positives.

11 Step 5: the LLM arm – the LLM proposes, the server decides

11.1 Why an LLM at all, and why this way

The grammar knows the SQL we wrote rules for. PostgreSQL master gets new planner features every few weeks (self-join elimination, eager aggregation, skip scan, OR-to-`ANY`, NOT NULL simplification, ...). Writing grammar for each by hand is slow, and the newest code is where regressions are. An LLM can read a commit (message, diff, the regression test it adds) and write grammar for it.

Two designs we rejected:

- **LLM as the generator** (Fuzz4All style): one model call per test case is 1,000× slower than

our 17–23 test cases per second, and the output is unchecked text with no derivation tree (so no oracles, no tree reduction).

- **LLM as the oracle** ("is this result correct?"): LLMs cannot evaluate SQL on our data reliably; every hallucinated "bug" costs triage time.

Our design keeps the model *off the hot path*: a few calls per commit produce a grammar fragment; the grammar then generates thousands of variations, each a typed tree the oracles can use. And nothing the model writes is trusted: a gate on the real server decides.

11.2 The pipeline

1. **Mine** (`src/pgfuzz/llm/commits.py`): `pgfuzz llm mine <pg clone> -since 2026-06-29 -until 26bd08fb -exclude bench/reinjected/commits.txt`. Keeps commits that touch query-processing code (optimizer, executor, parser, `utils/adt`, rewrite, nodes, ...) and skips typo/comment/pgindent/revert/warning commits. For each it stores a `CommitDigest`: sha, date, subject, body, files, changed function names, the SQL lines it added to the regression tests, and the **diff-coverage targets** (changed line numbers of every `.c` file).
2. **Prompt** (`prompt.py`). A system prompt that is the same for every commit and therefore **cached** (≈9,000 tokens): the `.pgg` format, every builtin with one line of documentation, the conventions (parenthesise operators, qualify columns, order-sensitive aggregates via `@sorted_agg`), the schema, the type list, and the whole current grammar. The user message is only the commit digest.
3. **Answer** in a strict JSON schema (structured output): `rationale` (which code paths changed and which SQL reaches them), `grammar_fragment` (new alternatives as variants of existing rules, plus new rules), 1–5 `seeds`, `features`, and 0–3 `rewrites` (equivalence templates for the metamorphic oracle).
4. **Gate** (`gate.py`, below).
5. **Repair**: a rejected proposal goes back to the model with the gate's findings – the parse error, the top server error messages with counts, which alternatives were never derived, which changed lines were not covered – up to 3 rounds.
6. **Ledger**: each round is one line of `ledger.jsonl` (commit, round, tokens in/out/cached, cost, full gate report). Resumable; the source of RQ4 (`pgfuzz llm report`).
7. **Use**: accepted fragments go to `results/llm/fragments/`, seeds to `results/llm/seeds/`, rewrites into the rewriter. The LLM arms of RQ1 are the grammar arm plus these files.

The client (`client.py`) has two modes. `api` calls Claude through the Anthropic SDK. `manual` writes the prompt to `prompts/<key>.md` and stops; an answer is saved to `responses/<key>.json` and the run continues – no API key needed. **The reported campaigns used manual mode with a script that sent each prompt to Google Gemini Flash models (free tier), because no Claude key was available;** the tool itself is unchanged. In both modes an existing response file is *replayed*: reruns are free and bit-for-bit reproducible, so the paper's numbers can be regenerated. Token usage is stored with every response.

11.3 The validation gate

A fragment for one commit is accepted only if:

1. **parse**: it parses together with the grammar, declares no new types and uses only known builtins;

2. **reachability**: when we sample 150 test cases with the fragment's alternatives boosted (weight ×25), they are derived in at least 5 statements (an alternative the engine can never complete is useless);
3. **validity**: at least 60% of the statements that use them run without error *on the server* – PostgreSQL decides, not the model;
4. **seeds**: seed test cases run; failing seeds are dropped;
5. **diff coverage** (on a gcov build): the fragment executes more of the commit's changed lines than the grammar without it, on the same sample size. Without a coverage build this is "not measured" and does not block.

The gate is also a small fuzzing campaign aimed at the commit: bug candidates hit while gating are recorded.

Diff coverage (`diffcov.py`) = changed lines executed / changed lines that are executable, from gcov counters. Each backend writes `.gcda` files when it exits, so the client reconnects before measuring. If the coverage build is at a later revision than the commit, the changed line numbers are mapped forward through `git diff`. All our builds are at the same commit (`26bd08fb`), so the mapping is the identity.

11.4 The rewrite gate (LLM-proposed metamorphic rewrites)

A rewrite is a template with `{e}` and a type, for example `CASE WHEN {e} IS NULL THEN NULL ELSE {e} END` for `int`. A wrong rewrite would make the metamorphic oracle report false positives forever, so the bar is higher:

1. **static**: known type, contains `{e}`, balanced parentheses, no volatile function;
2. **value probe**: evaluate `SELECT (template(v)) IS NOT DISTINCT FROM (v)` on the server for `v` = NULL, the type's edge values (min/max int, ±Infinity, NaN, −0, empty string/array, ...) and up to 6 data quantiles. One difference or error rejects it with the counterexample;
3. **live checks**: apply *only* this rewrite, with confirmation, to the expressions of 150 grammar test cases; accept only if it was applied at least 5 times, at most 20% of the rewritten queries errored, and **no** confirmed mismatch.

Measured (rewrite gate on master)

Rewrite	Template	Result
`null_case`	`CASE WHEN {e} IS NULL THEN NULL ELSE {e} END`	accepted, 90/90 equal
`substr_one`	`substr({e}, 1)` (text)	accepted, 88/88 equal
`plus_one`	`{e} + 1` (deliberately false)	rejected: 12 mismatches
`is_true`	`{e} IS TRUE` (bool)	rejected by the **value probe** at NULL

`e IS TRUE` is false, not NULL, when `e` is NULL. It passed 109 sampled live checks, because most predicates sit in `WHERE`, where NULL and false filter the same rows. Sampling is not proof; the probe was added because of this.

A confirmed mismatch is not thrown away: the gate stores the example with the note "not an identity, or a bug in PostgreSQL: triage the example". (BUG-02 is exactly a case where an identity, `e ≡ (SELECT e)`, was broken by PostgreSQL.)

11.5 Leakage and cost

- **No leakage into RQ3.** The miner excludes the 23 commits whose fixes we revert, so the model never sees the fix of a bug it is later scored on. The mining window ends at the benchmark base `26bd08fb`. (Training-data leakage of older public commits cannot be excluded; it is a threat to validity, discussed in the paper.)
- **Cost.** Prices in `client.py`; the cached system prompt is read at a tenth of the input price. Measured in the final run (Gemini Flash, free tier): 25 rounds, 328k input / 171k output tokens, 0 USD.
- **Status.** The pipeline is tested end to end with a live gate (it accepted a valid fragment, 251/251 statements valid, and rejected an invalid one with the server's own error message). The real run covered 10 of 25 mined commits (daily quota): 4 accepted, 16 of 25 rounds rejected on validity; results in the Results chapter.

11.6 RQ4: what we will report

Accepted / rejected fragments per gate check; repair rounds needed; diff coverage with vs without the fragment, per commit; tokens and dollars; the ablation of RQ1 (`llm_frag`: fragments only, `llm_seeds`: seeds only, `grammar_llm`: both) against `grammar_mm`; bugs whose test case used an LLM alternative (the tree records it).

Part IV
Evaluation

12 The re-injected bug benchmark (RQ3)

12.1 Why we need it

Counting "bugs found" in a live system is noisy: in a week of fuzzing each arm may find zero or one new bug, which supports no comparison. Klees et al. (CCS 2018) recommend ground truth: a set of known bugs, and the time each arm needs to find each one. Magma ports real bugs into programs. We do the same for PostgreSQL by **reverting the fixes** of recent real bugs on one master build: a mutation-testing view of the fuzzer (L13: inject faults, measure detection), but with real faults that real developers wrote and fixed.

12.2 Selection

From master, 31 July – 16 October 2026 (ending at our base commit `26bd08fb`), we took commits that (1) fix a bug in the planner, executor, or SQL-reachable functions; (2) add a regression test (our PoC); (3) revert cleanly together with the others on the base. 23 commits (`src/bench/reinjected/commits.txt`, `bugs.yaml`). `scripts/build_reinjected.sh` creates a worktree at the base, reverts the 23 fixes, and builds with assertions (optionally with coverage).

12.3 Servers and verification

Three kinds of server (`scripts/rq3_servers.sh`): **fixed** (the base), **solo**$_i$ (base with only fix i reverted, 23 builds), and **injected** (all 23 reverted; the campaign target). `pgfuzz bench verify` runs every PoC on all three and derives the symptom from the *difference* between fixed and solo$_i$ (not from what the commit message says):

Fix commit	Title	Symptom
11b56ee2	Fix bogus Assert in find_var_for_subquery_tle	assertion
94e1385e	Fix two edge cases in numeric log(base, val).	wrong_result
41180127	Fix jsonb_object_agg() and related aggs used as window functio	missing_error
b28a2b2b	Fix two errors in extraction of GIN indexquals from jsonpath q	wrong_result
f5635c27	Fix expr_is_nonnullable() for IS DISTINCT FROM	none
d8444163	Don't trust NOT NULL on virtual generated columns of partition	wrong_result
62e46c00	Fix ON CONFLICT inference for index expressions simplified to	error
738d19d4	Fix PlaceHolderVar placement for join alias expressions	internal_error
6b858a36	Don't reprocess PHV copies within outer aggregates' arguments	assertion
8b29b8da	Fix sorting on extra grouping keys of child relations	internal_error
d764d61a	Fix eager aggregation grouping on cross-type join keys	wrong_result
dfcf6fa7	Don't absorb child pathkeys into dummy AppendPaths for set-ops	internal_error
a464d31f	Fix relids of EC-derived join clauses for lateral UNION ALL me	assertion
852fd5b8	Fix join alias expansion in lateral TABLESAMPLE clauses	internal_error
a8bb1a77	Fix join removal when Vars reference the removed join via its	assertion
9d0adee8	Don't let JSON constructor coercions block SQL function inlini	none
895c1b5f	Fix duplicate enforcement of EC-derived conditions	none
35d3f206	Fix nestloop parameter handling for PlaceHolderVars in child j	internal_error
1bf74c80	Don't assume DISTINCT ON implies uniqueness when the tlist has	wrong_result
7774249d	Fix float8_regr_accum() for Inf/NaN with constant other input.	wrong_result
0d995793	Fix nullability check for a sub-select's upper-level Vars	wrong_result
d6eac691	Fix incorrect Result node flattening logic	wrong_result
3b120b1e	Fix integer to_char() overflow with V format	missing_error

Result. **20 of 23** have a symptom our oracles can see: 8 wrong results, 5 internal errors, 4 assertion failures, 2 missing errors (the fixed server raises an error, the buggy one does not), 1 wrong error. 3 change only plans, or need a user-defined operator, and are outside the denominator. All 20 show the same symptom with all 23 injected together, so the bugs do not mask each other. **Only 9 of 20** (assertions and internal errors) are visible to a crash-only fuzzer; the other 11 need a wrong-result or error oracle. This is the strongest argument for the oracle work.

The first verification said 12/23. Two of our mistakes: the PoC tables were named `i4`/`i8`, which our schema already uses for indexes, so `CREATE TABLE` failed (renamed to `rb_*`); and masked float aggregates hid 0 vs NaN (masking now keeps the NaN/Infinity class). Two PoCs depended on statistics and needed `ANALYZE`.

12.4 Attribution

Crash, assertion and internal-error bugs carry a signature (assertion file and expression, or normalised message) that the campaign matches *online* against the known-bug catalog (`bugs.yaml` doubles as it), so the time to first detection is exact. Wrong results are matched *offline*: a mismatch does not say which bug caused it. `pgfuzz bench attribute` replays every failure of a campaign on the fixed server and on each solo server: reproduces on solo$_i$ but not on fixed $\Rightarrow$ bug i; reproduces on fixed $\Rightarrow$ native (a real bug of the base or a false positive, goes to triage). In smoke run 2 this assigned an internal error to `rb-dfcf6fa7`, although `rb-8b29b8da` has the same message ("could not find pathkey item to sort"): online matching alone counted 2.

13 Experiment design

13.1 Research questions

RQ1 (reach) Does each step up the ladder (Random → Grammar → Grammar+Oracles → +LLM) increase valid-statement rate, line and branch coverage of the backend, SQL features, feature pairs, query shapes and plan shapes?

RQ2 (bugs and precision) Which bugs does each arm find, through which oracle, and what is each oracle's precision (the ledger)?

RQ3 (ground truth) How many of the 20 observable re-injected bugs does each arm find in 1 hour, and how fast?

RQ4 (LLM) What do the LLM fragments add (diff coverage of their commits, coverage, features, bugs), what does the gate reject, and what does it cost?

13.2 Arms

Arm	Generator settings
`random`	seeds from a small AST, 1–4 text mutations, 5% fresh random; no feedback
`grammar`	typed grammar, no metamorphic checks, no plan guidance
`grammar_mm`	+ metamorphic checks on 50% of queries, plan-shape guidance, bandit ($\eta = 0.1$), data profile (30%), RLS policies
`grammar_mm_flat`	as `grammar_mm` without the bandit (ablation, RQ1 only)
`llm_frag`	`grammar_mm` + accepted LLM fragments (RQ4 ablation)
`llm_seeds`	`grammar_mm` + LLM seeds (5% of test cases) (RQ4 ablation)
`grammar_llm`	`grammar_mm` + fragments + seeds (the full system)

RQ1 runs all 7 on a gcov build of master; RQ3 runs `random`, `grammar`, `grammar_mm`, `grammar_llm` on the injected server, with the cross-version executor (reference 18.6) for every arm.

Fairness rules.

- Same executor, oracles (plan variants, cross-version), schema, server build and budget for every arm; arms differ only in the generator (and in what it can attach: metamorphic checks need trees).
- Paired design: trial k has the same seed in every arm.
- 5 trials × 1 hour per arm (Klees et al. ask for ≥10 when possible; we will add trials if time allows), 4 workers in parallel, one server per worker.
- **One commit for everything**: the coverage build, the benchmark base and the end of the LLM mining window are all `26bd08fb`.
- **No mixed coverage**: each coverage server gets its own `GCOV_PREFIX`, otherwise parallel trials would merge counters.
- **No shared reference tables**: one database per worker on the 18.6 server.

13.3 Metrics

Valid-statement rate (server accepted / all), parse-valid rate; line and branch coverage of `src/backend` (lcov every 300 s); SQL features (catalogue of 148, from the statement text), feature pairs, distinct query shapes (statement with literals and names abstracted), distinct plan shapes; metamorphic

checks and plan-variant runs; unique bug candidates (by signature); known bugs reproduced and time to first detection; LLM tokens and cost. All are recorded on a timeline every 30 s, so we can plot growth curves.

13.4 Statistics

`src/pgfuzz/eval/stats.py`, pure Python (no SciPy):

- **medians and IQR** per arm (fuzzing results are skewed; means mislead);
- **95% bootstrap CI** of the median (2,000 resamples, fixed seed 5187, so a report is reproducible);
- **Mann–Whitney U**, two-sided; exact distribution without ties and $n \leq 20$, normal approximation with tie and continuity correction otherwise;
- **Vargha–Delaney** $\hat{A}_{12}$ with magnitude (negligible < 0.56, small, medium, large ≥ 0.71, symmetric below 0.5).

With 5 trials per arm the smallest possible two-sided exact p is $2/\binom{10}{5} = 0.0079$, so significance at 0.05 is reachable only with a clear separation; we report effect sizes for exactly this reason. With 1 trial (the smoke runs) $p = 1.0$ always: the smoke tables show a working pipeline, not evidence.

RQ3 time-to-bug is censored: a bug not found in 1 hour has time ">3600 s". We report per bug the number of trials that found it and the median time over the trials that did, plus the Mann–Whitney test on the per-trial number of bugs found.

14 Results so far

14.1 Final campaigns (29 Oct): RQ1–RQ4

Single source: `docs/RESULTS\_ANALYSIS.md`. 5 trials × 1 h per arm; RQ1/RQ2/RQ4 on one laptop, RQ3 on another. The LLM was Gemini Flash, not Claude.

Arm	Valid	Lines	Features	Plan
Random	26.0%	22.2%	23	
Grammar	99.3%	36.7%	101	
Grammar+Oracles	96.3%	36.2%	101	
+LLM (both)	95.3%	35.7%	106	

- **RQ1.** Grammar vs random: $\hat{A}_{12} = 1.00$, $p = 0.008$. Oracles: no line gain ($p = 0.42$), plan shapes ×1.7 ($\hat{A}_{12} = 0.84$, $p = 0.095$). LLM: +5 features ($p = 0.004$), no coverage gain.
- **RQ2.** 93 candidates: 7 real, 15 duplicates, 37 false positives, 28 flaky, 6 open. New bugs #19802 (confirmed, patch) and #19803 (confirmed, fixed on master); rediscovered #19800, #19772, #19784, #19771. Four new false-positive classes were fixed *after* the runs and not re-measured.
- **RQ3.** Best arm found 2 of 20 benchmark bugs; no wrong-result bug by any arm. The benchmark does not show an oracle advantage.
- **RQ4.** 4 of 10 commits accepted; the gate accepted one rewrite that changed the collation (now rejected). 0 USD.

The older smoke results below are kept as development history.

What is and is not a result yet

The two bugs and the component measurements below are real and reproducible. The smoke-run tables are single short trials: they show that the pipeline works end to end, not that one arm beats another ($p = 1.0$ with one trial). The RQ tables of the paper come from the overnight campaigns (Section 16).

14.2 BUG-01: JSON_TABLE does not assign collations

Found 16 October by the grammar campaign (`configs/cloud_hunt.yaml`, trial 0, test case 13306, about 600 s in) as an assertion failure on master:

```
TRAP: failed Assert("list_length(rcexpr->inputcollids) == nopers"),
      File: "execExpr.c", Line: 2078
```

Minimal reproducer.

```
-- expected t; 18.6 returns f, assert builds crash
SELECT * FROM JSON_TABLE(jsonb_build_object('k', ROW(1, 2) < ROW(3, 4)), '$'
       COLUMNS (a bool PATH '$.k'));
-- expected t; 18.6: ERROR: could not determine which collation to use ...
SELECT * FROM JSON_TABLE(jsonb_build_object('k', 'a' < 'b' COLLATE "C"), '$'
       COLUMNS (a bool PATH '$.k'));
```

Build	Behaviour
master `26bd08fb`, cassert	backend aborts on the assertion
18.6 release	**wrong result**: `ROW(1,2) < ROW(3,4)` evaluates to `false`
18.6 release	text comparison in the context item or in `PASSING`, even with `COLLATE "C"`: spurious collation error
17, 18, master	same code, all affected

Root cause. `transformJsonTable()` in `src/backend/parser/parse_jsontable.c` transforms the context item and the `PASSING` arguments with `transformExpr()` but never calls `assign_expr_collations()` on them (the column expressions get it; `XMLTABLE`'s document expression gets it in `parse_clause.c`). A function in `FROM` is not covered by the query-level collation pass, so the expressions keep `InvalidOid`. A `RowCompareExpr` keeps an empty `inputcollids` list; `ExecInitExprRec()` walks the operators *together with* that list, so it compiles **zero** column comparisons and the result is `false`. The assertion catches exactly that mismatch on assert builds; release builds silently return the wrong answer. A text comparison reaches `varstr_cmp()` with no collation and errors.

Fix (one line, patch with regression tests in `docs/bughunt/`):

```
  tf->docexpr = transformExpr(pstate, (Node *) jfe, EXPR_KIND_FROM_FUNCTION);
+ assign_expr_collations(pstate, tf->docexpr);
```

The new tests fail on unpatched 18.6 and pass with the patch; the SQL/JSON, json, jsonb, jsonpath and collate suites pass.

What it shows. An assertion build turns a silent wrong result of the release into a crash a fuzzer cannot miss – the reason we fuzz cassert builds. The grammar arm *rediscovered* it in smoke run 1 (4 minutes). Tree reduction gives a 79-token reproducer (Section 9.2).

14.3 BUG-02: range containment with a NULL element

Found 23 October by the new metamorphic oracle (rewrite `expr:scalar_sub`), in the first 150 test cases it ever ran. The generated query (abridged) contained `COALESCE('(,)'::int4range, '(,5]'::int4range) @> a4.c0` where `a4` is the right side of a `LEFT JOIN`. The rewrite replaced the typed node `expr<int4range> = '(,)'::int4range` by `(SELECT ('(,)'::int4range))`. The original returned `t` for the three NULL-extended rows, the rewrite returned NULL. Confirmed by re-running both sides.

Minimal reproducer.

```
SELECT c,
       '(,)'::int4range @> c         AS unbounded_contains,  -- expected NULL for c =
    NULL
       c <@ '(,)'::int4range         AS unbounded_contained, -- expected NULL
       'empty'::int4range @> c       AS empty_contains,      -- expected NULL
       NOT ('empty'::int4range @> c) AS not_empty            -- expected NULL
FROM (VALUES (NULL::int), (1)) v(c);
-- actual (17, 18.6, master): the NULL row gives  t | t | f | t
```

`EXPLAIN VERBOSE` shows `Output: true`: the planner replaced the expression by a constant. The operator itself is strict: `SELECT (SELECT '(,)'::int4range) @> NULL::int` returns NULL. (With a single-row `VALUES` the planner folds everything with the NULL constant and the bug hides; two rows or a column are needed.)

Root cause. Since PostgreSQL 17, the planner support functions of `elem <@ range` and `range @> elem` (`find_simplified_clause()` in `src/backend/utils/adt/rangetypes.c`) simplify a call with a constant range: for a bounded range they build `elem >= lower AND elem < upper` (NULL-correct), but for an empty range they return the constant `false`, and for `(,)` the constant `true`, ignoring the element. The operators are strict, so for a NULL element the answer must be NULL. Wrong wherever NULL and false/true are distinguished: the select list, `NOT`, `IS NULL`, `IS DISTINCT FROM`, `CASE`, `COALESCE`, outer-join and `NOT IN` logic. A silent wrong result in released versions 17 and 18.

Fix.

```
if (empty || (lower.infinite && upper.infinite))
{
    /* the operators are strict, so a NULL element yields NULL */
    if (!expr_is_nonnullable(root, elemExpr, NOTNULL_SOURCE_HASHTABLE))
        return NULL;                        /* do not simplify */
    return makeBoolConst(!empty, false);
}
```

With the patch the constant is still produced when the element is provably non-null. Full `make check` passes after updating `rangetypes.out`; two existing expected outputs change (`current_date <@ daterange 'empty'` is no longer folded, because `expr_is_nonnullable()` does not know `CURRENT_DATE` is never NULL) – we mention this to reviewers. Patches for master and for `REL_17`/`REL_18` are in `docs/bughunt/`.

Why it matters for the paper. It is our motivating example: (1) a silent wrong result in released PostgreSQL, no crash, no log entry; (2) a crash-only fuzzer cannot see it; (3) a string-level rewrite would not have found it: the oracle needed to know that this token span is *one typed expression node* of type `int4range`, so wrapping it in `(SELECT ...)` is an identity; (4) localization classified it as "not toggled by planner settings", pointing at a support function, which is the true cause.

14.4 Other candidates from the bug hunt

- **Crash-recovery failure** (unreproduced): after the BUG-01 crash chain one server could not restart: `FATAL: could not create file "base/5/165792": File exists`. We traced the file in the WAL to an unlogged sequence created in an open transaction and read `ResetUnloggedRelations`, but a clean data directory never reproduced it. Not reported; evidence kept.
- `ARRAY(SELECT)`, TLP-HAVING, numeric `DISTINCT`, clock, PoC replay, `corr`: our false positives / intended changes, all in the ledger (Section 10).

14.5 Smoke run 2 (RQ3 pipeline, 15 minutes per arm, injected server)

Arm	Test cases	Valid statements	Features	Plan shapes	Benchmark bugs
Random/Mutation	174,104	26.3%	17	22	0
Grammar	16,841	99.6% of 56,233	101	5,416	0
Grammar+Oracles	15,861	99.7% of 42,852	101	5,968	**1** (158 s)
Grammar+Oracles+LLM*	15,800	99.7% of 42,684	101	5,937	**1** (160 s)

*without fragments (the LLM extension is Phase 1), so it equals Grammar+Oracles, as expected. Evidence: `docs/evidence/rq3_smoke2/`.

How to read it:

- The random arm runs 10× more test cases (one short statement each, 193 statements/s) but only 26.3% are valid (33.8% even parse) and it reaches 17 SQL features and 22 plan shapes: speed is not reach.
- Grammar+Oracles reaches more plan shapes than plain Grammar (5,968 vs 5,416) with fewer test cases: plan-shape guidance and the bandit work.
- The bug found, `rb-dfcf6fa7` ("could not find pathkey item to sort", set operations over dummy Append paths), was found only by arms with the extra oracles and plan guidance. The online matcher counted 2 known bugs (the message is shared with `rb-8b29b8da`); attribution by replay picked the right one.
- The random arm's 2 candidates (an assertion the online matcher took for `rb-a8bb1a77`, and a crash) were the shared-server PoC replay: the `TRAP` came from another backend (fixed: PID check). Plain Grammar's 2 candidates were the `corr` cross-version difference (an intended master change, at 677 s) and an internal error "Append child's targetlist doesn't match Append" that never reproduced (flaky).

14.6 Full-system smoke on master (3 minutes, one worker)

3,166 test cases, 99.5% valid statements, 1,991 metamorphic checks of all kinds, 7,956 plan-variant runs, 0 false positives.

14.7 Component checks

Each new component was measured on master with a single run (the paper has the same table, "component checks"):

Component	Measured
Metamorphic oracles, first live run	600 test cases, 907 checks, 0 false positives; BUG-02 in the first 150
New rewrite kinds (TLP-agg, IN↔OR, negated comparison, BETWEEN, NOT NULL, ORDER BY)	798 live checks, 0 false positives
RLS leak oracle	532 checks, 0 false positives; the role sees 131 of 203 rows of `t0`
CERT	58 checks, 0 violations
Cross-version (master vs 18.6)	1,155 comparisons, 0 mismatches after fixes
Tree reduction (BUG-01 case)	590 → 79 tokens (token reduction alone: 116)
Token reduction (BUG-02 case)	4 statements / 107 tokens → 1 / 36 in 307 runs
Localization	BUG-02 → "not toggled by planner settings" (support function: correct)
Bandit weights	533 vs 419 distinct plan shapes per 1,000 test cases, same validity
Data profile (`@dlit`)	predicates selecting 5–95% of rows: 29.3% → 58.6%
LLM rewrite gate	2 identities accepted (90/90, 88/88); `e+1` rejected (12 mismatches); `e IS TRUE` rejected by the value probe
LLM fragment gate	valid fragment accepted (251/251 statements valid); invalid rejected with the server's message
Tree replay	599 of 600 stored trees replay to identical SQL in a fresh process
Benchmark	20/23 observable, same symptom injected together

14.8 Classes of false positives we found and fixed

This list is RQ2 material and a likely professor question ("how do you know your oracles are right?"):

1. **Unordered collection**: `ARRAY(SELECT ...)` without `ORDER BY` → element order is plan-dependent → excluded by `eligible()`.
2. **Equal values, different text**: `DISTINCT`/`GROUP BY` over `numeric` keeps `21.00` or `21.0000` → canonical numeric scale in digests; TLP with `DISTINCT` only over unique-text types.
3. **Rewrite changed semantics**: removing `HAVING` from a level without `GROUP BY` → `HAVING TRUE`.
4. **Scope**: NOT NULL conjunct with a column from a nested sub-select → `_nested_under` (found by tests before any campaign).
5. **Environment**: clock in seed data, sequences across servers, shared reference database → deterministic data, prelude, per-worker databases.
6. **Harness**: a foreign crash blamed on the current statement → PID-checked evidence.
7. **Intended change**: `corr` on master → `_XVERSION_CHANGED` list.
8. **Over-masking** (a false *negative*): float masking hid 0 vs NaN → masked floats keep their class.

Part V
Paper, plan and demo

15 The paper

ACM/FSE format (`paper/`, `make` builds `main.pdf`; the current draft is `docs/PAPER_DRAFT.pdf`). Numbers that come from the campaigns are written with `\num{}` and appear in blue until filled.

`make check` reports LaTeX errors, undefined references and banned words.

15.1 The five graded parts and where they are

Graded part	Paper section	Content
Motivating example	Introduction	BUG-02: a silent wrong result in released PostgreSQL (17, 18), invisible to a crash fuzzer, found by a typed tree rewrite that a string rewrite could not have made safely
Workflow	Approach (Steps 1–5) + figure	typed grammar → tree rewrites as oracles → run several ways → classify/triage/reduce → LLM extension behind a gate
Research questions	Evaluation	RQ1 reach, RQ2 bugs and precision, RQ3 re-injected bugs, RQ4 LLM; component checks table
Related work	Related Work	SQLancer (PQS, NoREC, TLP), QPG, CERT, EET, RAGS, SQLsmith, SQUIRREL, SQLRight, AFLGo, Fuzz4All, Magma, Klees, ddmin, HDD, Vargha–Delaney, Arcuri–Briand: 18 references, each checked against the publisher, DBLP or author pages
Threats to validity	Threats	internal (randomness, our own oracles, verdicts by us, rewrite gate is testing not proof, attribution), external (one system, one commit, one schema, recent bugs, one model), construct (coverage $\neq$ detection, symptom mismatch, gate $\neq$ testing the behaviour)

Other sections: background, problem statement (what a wrong-result oracle needs; why recent commits), implementation, discussion (oracle precision as the bottleneck; what the LLM is and is not good at), conclusion, data availability, use of generative AI (the LLM is part of the method; we disclose its role and store every prompt and response).

Status. All number-free text is written; the main text is about one page over the 12-page limit, to be cut once the result tables exist.

15.2 Contributions as we state them

1. **One tree, four jobs**: a typed SQL grammar whose derivation trees are used for generation, mutation, metamorphic oracles (12 kinds, applied at any query level) and failure reduction.
2. **Measured soundness**: generator validity and per-oracle precision before and after each false-positive class was fixed.
3. **A gated LLM extension**: the model reads recent commits and proposes grammar and rewrites; a server-side gate (parse, reach, validity, diff coverage; value probe and live checks for rewrites) decides.
4. **A ground-truth benchmark** of 23 real PostgreSQL bugs re-injected into one build, with fixed/solo/injected servers and replay attribution.
5. **Two previously unknown PostgreSQL bugs**, with patches.

16 Execution plan and timeline

`docs/EXECUTION_PLAN.md` has the exact commands, expected times, what to look at and what to send back for every phase.

Phase	What	Machine time	Our time
0	builds (master cassert, 18.6, gcov build, injected, 23 solo) + reference server	∼60 min	5 min
1	LLM arm: mine commits, extend grammar (RQ4 ledger)	1–2 h	10 min
2	RQ3: verify, campaign (4 arms × 5 × 1 h), attribute	∼5.5 h	15 min
3	RQ1/RQ2/RQ4: coverage campaign, 7 arms × 5 × 1 h	∼9 h	10 min
4	triage every candidate, precision table	1 h	30 min
5	reports, figures, tables into the paper	10 min	20 min

Machine: Linux/WSL2, 4+ cores, 16 GB RAM, ∼25 GB disk. Phases 2 and 3 run overnight and resume if interrupted.

When	What
24–26 Oct	Phases 0–1 (builds, LLM extension on Gemini Flash, free tier)
26–27 Oct	Phase 2: RQ3 overnight
27–28 Oct	Phase 3: RQ1/RQ2/RQ4 overnight
28–30 Oct	triage all candidates; **file BUG-01 and BUG-02 upstream**
Meeting 3 (2–6 Nov)	all RQ tables and figures; paper with numbers
12 Nov	presentation
13 Nov	paper and repository

Risks and fallbacks.

- New false-positive classes in long campaigns: triage handles them; they are RQ2 data, not a failure of the project.
- LLM API unavailable (this happened): manual mode with Gemini Flash answers; stored responses replay.
- Upstream does not answer before 13 Nov: we report the filing, the reproducer and the patch; PostgreSQL developers often reply within days for reproducible wrong results.
- A build or revert fails on a team machine: the scripts print what to send; a shallow clone is the usual cause (`git fetch -deepen`).

17 Live demo script

Five minutes, on a machine with master running on port 5433 (or any server from `scripts/pg_server.sh`). Rehearse once; keep the outputs as screenshots in case the network or server fails.

1. **Generate** (30 s): `cd src && python -m pgfuzz sample configs/local.yaml -n 3`. Show generated test cases (setup statement, then queries): every column qualified, every operator parenthesised, typed literals.
2. **Validity** (30 s): `python -m pgfuzz sample configs/local.yaml -n 300 -execute`: valid rate ≈99.5%, top errors, worst alternatives, bug candidates printed as they happen.
3. **BUG-02 live** (1 min): paste the minimal reproducer into `psql` on 18.6 and on master: the NULL row returns `t`. Then `EXPLAIN VERBOSE`: `Output: true`. Then the sub-select form: NULL.
4. **Triage** (1 min):

```
python -m pgfuzz triage configs/local.yaml \
  ../docs/evidence/tree_reduction/original.sql
```

– watch the tree stage, then statements and tokens; open `report.md` and `upstream_report.txt` in `../results/triage/<signature>/`.

5. **Precision** (30 s): `python -m pgfuzz precision ../triage/verdicts.yaml`: the RQ2 table.
6. **Reports** (1 min): open `docs/evidence/rq3_smoke2/report.md` (medians, IQR, $\hat{A}_{12}$, time to first bug) and `bench/reinjected/verify.json`.

Check the exact flags with `python -m pgfuzz <cmd> -help` before the meeting; the appendix lists the commands.

Part VI
Questions the professor may ask

18 Questions and answers

Short answers we can say out loud, with the section that has the details.

18.1 Motivation and novelty

Q: What is new here compared with SQLancer?

SQLancer's oracles (TLP, NoREC, PQS) run on queries from its own small generators, whose structure it knows. We apply the same and more oracles to *derivation trees of a large typed grammar* (78 rules, 421 alternatives), at any level of a nested query, and we use the same tree for mutation and for reduction. Two further things SQLancer does not do: steering towards recent commits with a gated LLM, and measuring oracle precision with a ledger. And a ground-truth benchmark of 23 re-injected PostgreSQL bugs to compare generators fairly.

Q: Why not just use SQLsmith or SQLancer as the baseline?

The paper compares *generators* under identical oracles and budget; plugging an external tool into our executor would change two things at once. Our random/mutation arm is the standard unguided baseline (AFL-style text mutation). We discuss SQLancer, SQLsmith and SQLRight in related work, and the motivating example (BUG-02) is a bug that had not been reported although it has been in released PostgreSQL since 17.

Q: Why PostgreSQL and why master?

PostgreSQL is the most used open-source DBMS; its master branch gets new planner optimisations every month, and regressions there are fixed fastest ("open items" before a release), which helps our chance of an accepted bug. We also check every finding on the 18.6 release: BUG-01 and BUG-02 affect released versions.

Q: Why do wrong results matter more than crashes?

A crash is loud: the user sees an error and the log has a trace. A wrong result is silent: an application gets wrong rows and nobody notices. 11 of the 20 observable benchmark bugs are invisible to a crash-only fuzzer (Section 12).

Q: In one sentence, what is "one tree, four jobs"?

The same typed derivation tree generates the SQL, is mutated (regenerate, splice, insert, drop subtrees), is rewritten by the oracles into an equivalent query, and is shrunk when a test case fails; every job reuses the grammar's knowledge of types and scopes.

18.2 Grammar and generator

Q: How did validity go from 83% to 99.5–100%?

By a measurement loop: `pgfuzz sample -execute` prints the valid rate, top server error messages and the alternatives with the worst valid rate; each fix targets one rule (type a function's arguments correctly, keep a column in scope, avoid a million-row `generate_series`). Four rounds took it from 83.4% to 100% on 966 statements (Section 4). With the larger grammar today it is 99.5–99.7%.

Q: Why is the grammar typed? Isn't a CFG enough?

A CFG gives syntactically valid SQL that is mostly rejected by the analyzer: `int + text`, unknown columns, aggregates in `WHERE`. Types and scopes make generation a proof search for $\Gamma \vdash e : T$ (L29–30), so 99.5% of statements reach the planner and executor, which is where the bugs are. Types are also what makes rewrites sound: `e+0` is an identity for `int`, not for `float8` ($-0 + 0$ prints 0).

Q: What is the derivation tree exactly?

A node per rule expansion, recording which alternative was chosen, the bound type, and for builtins the chosen value (a column, a literal). Its leaves read left to right are the SQL. The renderer wraps every node's tokens in invisible markers, so we know which token span each node produced.

Q: How does replay work, and why is it needed?

Given a stored tree, the engine re-derives it following the same choices, but re-binds names and types in the current state. Mutation and reduction edit a tree and replay it, so the result stays valid even if aliases shift. Trees are stored as JSON with stable alternative ordinals; 599 of 600 test cases replay to identical SQL in a fresh process.

Q: What does the bandit do and is it significant?

Multiplicative weights over grammar alternatives: reward 1 if the test case entered the corpus (new signature, feature, feature pair or plan shape), -0.5 if the statement failed; weight $\times e^{\eta(r-\bar{r})}$, $\eta = 0.1$, clipped to $[0.2, 5]$. In one 1,000-test-case run: 533 vs 419 plan shapes at the same validity. One run is not significance; RQ1 has an ablation arm (`grammar_mm_flat`) with 5 trials.

Q: What is the data profile for?

Constants chosen blindly make predicates select all rows or none, so the planner never faces a hard choice. The profile gives per-column min/max/quantiles; `@dlit` draws a literal from the column being compared. Predicates selecting 5–95% of rows went from 29.3% to 58.6%. Boundary literals (min±1, -0, NaN, ±Infinity, empty array) are interval analysis applied to test inputs (L19–20).

Q: How big is the schema and why so small?

Nine relations, about 200 rows each, deterministic data with NULLs, all index kinds, partitioning, inheritance, a view, generated columns. Small tables keep each query fast (17–23 test cases per second with all oracles), and most optimizer bugs need structure (an index, a partition, a NULL), not volume. Volume-dependent bugs are an external threat.

18.3 Oracles

Q: How do you know a metamorphic mismatch is a real bug and not your rewrite?

Three layers. (1) Each rewrite has a soundness argument and is restricted by the tree to where it holds (Section 8.6): multiset contexts only, unique-text types for `DISTINCT`, no outer joins for the NOT NULL conjunct, etc. (2) A mismatch is confirmed by re-running both sides. (3) Triage reduces it and a human decides; every verdict goes into the ledger. Where we were wrong (TLP-HAVING, numeric scale), we fixed the class and added a test, and we report the precision before and after.

Q: Explain TLP in one breath.

For any predicate p, every row makes p TRUE, FALSE or NULL, exactly one of them, so the rows of a query equal the union of the rows with `WHERE p`, `WHERE NOT p` and `WHERE p IS NULL`. The optimizer treats the four queries very differently, so a bug in one shows as a mismatch.

Q: Explain NoREC.

`SELECT count(*) FROM f WHERE p` lets the optimizer use p (indexes, pushdown, join pruning). `SELECT sum(CASE WHEN p THEN 1 ELSE 0 END) FROM f` forces it to evaluate p on every row. Both count the rows where p is TRUE.

Q: Why did TLP-HAVING give false positives?

Without `GROUP BY`, a `HAVING` clause makes the level a single-group aggregate: it returns one row even if `WHERE` removes every row. Our left side *removed* `HAVING`, turning it into a plain query: 0 rows vs 1. Now the left side uses `HAVING TRUE`. It is a good example of a rewrite that is correct for the textbook case and wrong in a corner the larger grammar reaches.

Q: Why do you mask floats? Doesn't that hide bugs?

Summing floats in a different order (hash vs sort aggregation, parallel workers) legitimately changes the last digits, so exact comparison would flood us with false positives. We mask only when the query aggregates or uses windows, and the mask keeps the class (NaN, ±Infinity, finite): when masking everything we lost a real benchmark bug (0 vs NaN), which taught us this.

Q: What does the plan-variant oracle catch that metamorphic rewrites do not?

Bugs in one planner path or executor node: a join method, hash aggregation spilling with tiny `work_mem`, parallel plans, partition-wise join, memoize, eager aggregation. The rewrite oracles catch bugs in transformations that are always applied (no setting turns them off), like BUG-02. Localization uses both together.

Q: What does the cross-version oracle add?

An optimisation new on master that is wrong under every planner setting is invisible to plan variants but disagrees with 18.6, which does not have it. It needs exclusions for clock functions and intended changes (`corr` on master).

Q: What is the RLS oracle and why is it "security"?

The query runs as a restricted role with row-level security policies, and as superuser on explicitly filtered tables; both must be equal. More rows on the restricted side is a confidentiality leak: the policy is a label and the check is non-interference on concrete data (L21–22). RLS bugs have had CVEs in PostgreSQL (e.g. CVE-2023-2455, CVE-2024-10976). 532 checks, 0 false positives so far.

Q: What is CERT?

Cardinality Estimation Restriction Testing (Ba & Rigger, ICSE 2024): adding a `WHERE` must not raise the planner's row estimate. A violation is a *performance* outcome, reported separately, never as a wrong result.

Q: Which queries are excluded from comparison and why?

Those with no single correct multiset: `LIMIT/OFFSET` without full ordering, volatile functions, order-dependent windows, order-sensitive aggregates without `ORDER BY`, `ARRAY(SELECT)` without `ORDER BY`, `DISTINCT ON`, row locking, DML. The grammar writes order-sensitive aggregates with their own `ORDER BY`, so most queries stay eligible.

Q: How many metamorphic checks run per second?

In the full-system smoke, 1,991 checks in 180 s on one worker (about 11/s), alongside 7,956 plan-variant runs.

18.4 Triage and bugs

Q: Walk us through BUG-02.

Section 14.3. Short: since PostgreSQL 17 a planner support function simplifies `range @> elem` with a constant range; for an empty or unbounded range it returns the constant false/true and forgets that a NULL element must give NULL. Our expression rewrite wrapped the range in `(SELECT ...)`, which blocks the simplification, and the results differed for NULL-extended rows of a left join. Fix: only simplify when the element is provably non-null.

Q: Walk us through BUG-01.

Section 14.2. JSON_TABLE's context item and PASSING arguments never get collations assigned, so a row comparison inside them compiles zero column comparisons: false on 18.6, an assertion on master; text comparisons error. One-line fix: `assign_expr_collations`.

Q: Have you reported them? Why not yet?

Both have reproducers, root causes, patches and regression tests in `docs/bughunt/`. We are filing them this week (Phase 4 of the plan) after re-running on the newest 17.x/18.x/master and searching the archives for duplicates (none found on 23 Oct). We wanted the patches tested with the full regression suite first, which is done.

Q: Is BUG-02 really a bug, or intended?

The operators are declared strict, and without the planner simplification (the sub-select form) PostgreSQL itself returns NULL. The same expression giving different answers depending on whether a sub-select surrounds a constant cannot be intended.

Q: Why reduce on the tree if you have delta debugging?

Token reduction gets stuck: removing any single token breaks the SQL, while a smaller valid query needs a *different* derivation. Tree reduction replaces a subtree by the smallest derivation of the same rule and type, and replay keeps it valid. On the BUG-01 case: 79 tokens vs 116 for token reduction alone, at a similar number of server runs (555 vs 406).

Q: How do you reduce a metamorphic pair without breaking the equivalence?

The two queries share a prefix and suffix; we reduce the shared skeleton with a hole where they differ, and fill the hole with each side's middle. Every edit applies to both sides at once.

Q: How do you deduplicate?

Signatures, strongest first: top-5 stack frames, assertion file and expression, SQLSTATE + normalised message, oracle/variant + statement features, and a weak signal-only key. The strength is logged because counting by signature is a threat to validity.

Q: What is your oracle precision?

18 triaged candidates so far: 3 real (BUG-01, BUG-02, one re-injected bug), 13 false positives or intended changes, 2 flaky. Raw precision 0.19, but all 13 came from 6 classes we have since fixed, and since then 3,000+ new-kind, RLS, CERT and full-system checks gave 0 false positives. The long campaigns will give the per-oracle numbers for the paper (Section 10).

Q: What happens to the unreproduced recovery failure?

It stays in the evidence folder; we report only what we can reproduce. If it reappears in a campaign, triage will have a fresh chance with the full log.

18.5 LLM arm

Q: Why do you need an LLM at all?

To keep the grammar current with code that changes weekly. The model reads a commit and writes grammar that reaches the changed code. Doing that by hand for 30 commits would take days.

Q: What if the LLM hallucinates?

It cannot hurt the campaign: a fragment must parse, be derivable, run on the server at $\geq$60% validity, and (with a coverage build) execute more of the commit's changed lines. A rewrite must pass a value probe (NULL, type edges, data quantiles) and 150 live test cases with no mismatch. Rejections go back to the model with the server's messages, up to 3 rounds; everything is logged.

Q: Did the LLM ever propose something wrong that the gate caught?

Yes, by construction in our tests: `e + 1` was rejected with 12 mismatches; `e IS TRUE` passed 109 sampled live checks but the value probe rejected it at NULL. That is why the probe exists: sampling is not proof.

Q: Isn't there data leakage in RQ3?

The miner excludes the 23 fixes we revert, so the model never sees them in the prompt. Older public commits may be in its training data; that is a threat to validity we state. The mining window ends at the benchmark base commit.

Q: How much does it cost?

The final run used Gemini Flash on the free tier: 0 USD for 25 rounds (328k input, 171k output tokens, 10 commits because of the daily quota). A Claude run would cost a few dollars at published prices.

Q: What is diff coverage?

Changed executable lines of the commit that the generated SQL executed, divided by all changed executable lines, measured with gcov on a coverage build at the same commit. It tells us whether a fragment reaches the code it was written for.

Q: Why did the +LLM arm in the smoke run equal Grammar+Oracles?

It ran without fragments: the extension run is Phase 1 of the plan. It is a correctness check of the pipeline: the arm must behave like its base.

18.6 Evaluation and statistics

Q: Why 5 trials? Klees says more.

Budget: 7 arms × 5 trials × 1 hour is already 9 hours on 4 cores. We report medians, IQR, bootstrap CIs, Mann–Whitney p and $\hat{A}_{12}$, so the reader sees effect sizes even where p is not small. With 5 vs 5 the smallest exact two-sided p is 0.008. If time allows we add trials.

Q: Why Mann–Whitney and not a t-test?

Fuzzing metrics are skewed and bounded (time to bug is censored at the budget); a rank test makes no normality assumption. Arcuri & Briand recommend it with $\hat{A}_{12}$ for randomized algorithms.

Q: How do you handle bugs not found within the budget?

Time-to-bug is censored at 3600 s. We report, per bug, in how many trials each arm found it and the median time among those trials, and test the per-trial count of bugs found.

Q: Is the comparison fair to the random arm?

It gets the same server, oracles, schema, budget and seeds. It runs 10× more test cases (they are short) and still reaches 17 features vs 101. Its weakness is validity: 26% of its statements run. That is the point of the comparison.

Q: Why is your benchmark valid ground truth?

The bugs are real PostgreSQL bugs, fixed by PostgreSQL developers with regression tests. We verify that each PoC shows a symptom on its solo server and not on the fixed server, and the same symptom with all 23 injected (no masking). We count only the 20 observable ones.

Q: What if two injected bugs interact?

Verify shows no masking for the PoCs. Attribution replays each finding on each solo server; a finding that needs two bugs together is “unattributed” and not credited to any arm.

Q: How is coverage measured with 4 parallel trials?

Each worker has its own coverage server with its own `GCOV_PREFIX`, so counters never merge; lcov captures every 300 s.

Q: What are the threats to validity?

One system, one commit, one schema; benchmark of recent fixes; our own oracles and verdicts; the rewrite gate is testing not proof; one LLM at one time; coverage is not detection; attribution by replay (Section 15).

18.7 Course connection and process

Q: Which course concepts does the project use?

Grammar fuzzing and black/grey-box testing (L14–16), coverage and mutation testing (L13: the re-injected benchmark), soundness/precision (L2–L5: generator validity, oracle precision), type systems (L29–30: typed grammar), interval and constraint-based analysis (L19–20, L27–28: boundary and selective literals), information flow (L21–22: RLS oracle), Hoare-style assertions (each rewrite is an equivalence assertion), ASTs/CFGs (derivation trees, gcov). Table in Section 5.

Q: Who did what?

Venu: recent PostgreSQL features, campaigns, new grammar rules. Bhuvan: first pass on every bug candidate (reproduce, shrink, decide). Gowtham: experiment design, statistics, LLM arm, paper. All three know the whole system; this guide is our shared script.

Q: How did you use AI tools?

The LLM arm uses Google Gemini Flash models as part of the method (fragments, seeds, rewrites; every prompt and response stored; we had no Claude key). We used Claude Code as coding and drafting assistant. We also used an AI coding assistant during development; all code is tested (148 unit tests) and every reported number comes from a run in the repository. The paper's AI statement discloses both.

Q: What would you do with two more months?

More trials and longer campaigns; mutation of the schema and data during a campaign; more oracles from the plan-variant × rewrite product (e.g. rewrites under each variant); LLM-proposed oracles beyond expression identities; other DBMSs with the same grammar engine (only the grammar and executor are PostgreSQL-specific).

Q: What is the single biggest lesson?

Oracle precision, not generator power, limits a wrong-result fuzzer: a richer grammar finds the corners where two "equivalent" queries are not equivalent. Measuring that precision, and fixing each false-positive class with a test, is what makes the reports trustworthy.

Part VII
Appendix

A Glossary

Alternative
one right-hand side of a grammar rule, with a weight.

Arm
one generator configuration in the experiment (random, grammar, . . .).

Attribution
deciding which re-injected bug a failure belongs to, by replay on solo and fixed servers.

Bug candidate
a test case whose outcome is crash, assertion, sanitizer, panic, corruption, internal error or wrong result.

cassert
PostgreSQL built with `-enable-cassert`: `Assert()` checks are active.

CERT
cardinality-estimation restriction testing; performance oracle.

Check
one metamorphic comparison (left side, right side, kind).

Confirmed mismatch
both sides re-run and gave the same digests again.

Culprit
the statement of a test case with the most severe outcome.

Derivation tree
the record of which alternative each rule expansion chose; its leaves are the SQL.

Diff coverage
changed lines of a commit that were executed / changed executable lines.

Digest
order-insensitive hash of a result multiset after canonicalisation.

Eligible
a query whose correct result is a single multiset.

Feature
a SQL construct from a catalogue of 148 (e.g. `join:lateral`, `agg:filter`).

Fragment
grammar text proposed by the LLM for a commit.

Gate
the server-side checks a fragment or rewrite must pass.

GUC
a PostgreSQL configuration setting (“grand unified configuration”).

HDD
hierarchical delta debugging.

Injected / solo / fixed server
all 23 / one / none of the benchmark fixes reverted.

Ledger
(1) the precision ledger of triage verdicts; (2) the LLM ledger of rounds.

Localization
re-running a reduced wrong result under every plan variant to point at the responsible code.

Metamorphic oracle
compares a query with a transformed query whose result must be equal.

MINIMAL marker
tells the engine to derive a subtree with its smallest alternatives (tree reduction).

Plan shape
digest of the node types of a query plan.

Plan variant
a set of planner settings under which a query is re-run.

PoC
proof-of-concept SQL that shows a benchmark bug.

Replay
re-deriving a stored tree, re-binding names and types.

RLS
row-level security.

Signature
deduplication key of a failure.

Support function
a planner hook of a SQL function that can simplify or estimate calls to it (BUG-02 lives in one).

TLP
ternary logic partitioning.

Validity
share of generated statements that the server runs without error.

B Commands

All from `src/` unless noted.

```
python3 -m pytest -q                                    # 138 tests
python -m pgfuzz validate grammar/postgres.pgg          # check a grammar / config
python -m pgfuzz sample configs/local.yaml -n 5         # print test cases
python -m pgfuzz sample configs/local.yaml -n 300 --execute   # validity loop
python -m pgfuzz selftest configs/rq1.yaml              # servers reachable, oracles on
python -m pgfuzz run configs/rq1.yaml -j 4              # campaign (resumable)
python -m pgfuzz report ../results/rq1                  # tables, plots, statistics
python -m pgfuzz replay configs/local.yaml tc.sql --repeat 3
python -m pgfuzz triage configs/local.yaml <failure dir> [--no-tree] [--reference ...]
python -m pgfuzz precision ../triage/verdicts.yaml [--latex]
python -m pgfuzz profile configs/local.yaml             # data profile -> corpus/profile.
    json
python -m pgfuzz features file.sql --missing            # SQL features of statements
python -m pgfuzz llm mine <pg clone> --since 2026-06-29 --until 26bd08fb \
        --exclude bench/reinjected/commits.txt
python -m pgfuzz llm extend configs/rq1.yaml ../results/llm/commits.json [--mode manual]
python -m pgfuzz llm report
python -m pgfuzz bench verify configs/rq3.yaml
python -m pgfuzz bench attribute configs/rq3.yaml ../results/rq3
# repository root:
bash scripts/setup_env.sh            # master (cassert) + REL_18 builds
bash scripts/build_reinjected.sh     # base + 23 reverted fixes
bash scripts/rq3_servers.sh build-solo | init 4 | start 4
bash scripts/rq1_servers.sh build | init 4 | start 4 | coverage <w>
bash scripts/ref_server.sh init | start   # 18.6 reference on port 5435
bash scripts/bisect_pg.sh GOOD BAD reduced.sql crash|error|output|differ
```

C File index

File	What it is
src/grammar/postgres.pgg	the typed grammar (628 lines)
src/pgfuzz/grammar/dsl.py	parser of the .pgg format
src/pgfuzz/grammar/engine.py	derivation engine: scopes, types, markers, replay, MINIMAL
src/pgfuzz/grammar/generator.py	test cases, corpus, mutation, bandit, profile literals, failure trees
src/pgfuzz/grammar/rewrite.py	the 12 metamorphic check kinds
src/pgfuzz/grammar/profile.py	data profile and type edge values
src/pgfuzz/grammar/treeio.py	tree JSON with stable ordinals
src/pgfuzz/reference/pg_executor.py	executor: savepoints, log, variants, checks, CERT, PID filter
src/pgfuzz/reference/xversion_executor.py	master vs 18.6
src/pgfuzz/eval/differential.py	variants, eligible, digests, float masking
src/pgfuzz/eval/oracle.py	classification into outcomes
src/pgfuzz/eval/signature.py	deduplication
src/pgfuzz/eval/features.py	SQL feature catalogue (148), query shapes
src/pgfuzz/eval/stats.py	median, IQR, bootstrap CI, MWU, $\hat{A}_{12}$
src/pgfuzz/triage/triage.py	reproduce, reduce, observe, report, upstream draft
src/pgfuzz/triage/reduce.py, sqltok.py	ddmin, HDD, pair reduction, SQL tokenizer
src/pgfuzz/triage/tree_reduce.py	tree reduction
src/pgfuzz/triage/localize.py	localization and suspect files
src/pgfuzz/triage/precision.py	precision ledger
src/pgfuzz/llm/commits.py	commit miner
src/pgfuzz/llm/prompt.py	system prompt, response schema
src/pgfuzz/llm/client.py	API / manual / replay client, prices
src/pgfuzz/llm/gate.py	fragment gate, rewrite gate, value probe
src/pgfuzz/llm/diffcov.py	diff coverage from gcov
src/pgfuzz/llm/loop.py	propose–gate–repair loop, ledger
src/pgfuzz/bench/reinjected.py	benchmark verify and attribute
src/bench/reinjected/	bugs.yaml, commits.txt, pocs/, verify.json
src/corpus/	schema (SQL, YAML), RLS policies, data profile
src/configs/	rq1.yaml, rq3.yaml, local.yaml, hunt configs
triage/verdicts.yaml	verdicts of all triaged candidates
docs/bughunt/	BUG-01, BUG-02 reports and patches, hunt log, evidence
docs/evidence/	tree reduction, RQ3 smoke 2
docs/EXECUTION_PLAN.md	commands for all campaigns
docs/PLAN_COVERAGE.md	meeting-1 plan item → code → measurement
docs/PROGRESS_MEETING2.md	progress report
docs/meeting2/	this guide and the slides
paper/	the paper